



MATHEUS GUSTAVO COPPI DA SILVA

**AVALIAÇÃO COMPARATIVA DE DESEMPENHO ENTRE REST E GRPC EM
APLICAÇÕES DISTRIBUÍDAS**

MATHEUS GUSTAVO COPPI DA SILVA

**AVALIAÇÃO COMPARATIVA DE DESEMPENHO ENTRE REST E GRPC EM
APLICAÇÕES DISTRIBUÍDAS**

Trabalho de Conclusão de Curso apresentado ao curso de Sistemas de informação do Centro Universitário Avantis de Balneário Camboriú para a obtenção do título de Bacharel em Sistemas de informação.

Orientador: Prof. Luiz Fernando Arruda, Msc

RESUMO

A crescente complexidade dos sistemas e a demanda por escalabilidade impulsionaram a adoção da arquitetura de microsserviços em detrimento do modelo monolítico. Essa abordagem favorece a modularização, a escalabilidade granular e a autonomia das equipes de desenvolvimento, mas também impõe desafios operacionais e de observabilidade. Neste Trabalho de Conclusão de Curso, realiza-se uma comparação entre os protocolos de comunicação *REST* e *gRPC* em arquiteturas baseadas em microsserviços, utilizando métricas de observabilidade como latência (P95 e P99), *throughput* e uso de recursos. A metodologia envolve a implementação de dois serviços equivalentes, instrumentação com Prometheus, execução de testes de carga e análise estatística dos resultados. O objetivo é identificar o protocolo mais eficiente em diferentes cenários de uso. Os resultados esperados visam fornecer subsídios técnicos para orientar decisões arquiteturais em sistemas distribuídos, promovendo um equilíbrio entre desempenho e flexibilidade.

Palavras-chave: microsserviços. *REST*. *gRPC*. *observabilidade*

ABSTRACT

The increasing complexity of systems and the demand for scalability have driven the adoption of microservices architecture over the monolithic model. This approach promotes modularization, granular scalability, and development team autonomy, but also introduces operational and observability challenges. In this Undergraduate Thesis, a comparison is conducted between the communication protocols *REST* and *gRPC* in microservices-based architectures, using observability metrics such as latency (P95 and P99), *throughput*, and resource usage. The methodology involves implementing two equivalent services, instrumenting them with Prometheus, conducting load tests, and performing statistical analysis of the results. The objective is to identify the most efficient protocol in different usage scenarios. The expected results aim to provide technical insights to guide architectural decisions in distributed systems, promoting a balance between performance and flexibility.

Keywords: microservices. *REST*. *gRPC*.

LISTA DE ILUSTRAÇÕES

Figura 1 – Estrutura típica de arquitetura monolítica.	17
Figura 2 – Arquitetura Orientada a Eventos, ilustrando a comunicação assíncrona entre microserviços através de um barramento de eventos. . .	19
Figura 3 – Arquitetura Serverless típica, mostrando os componentes principais: <i>API Gateway</i> , Funções como Serviço (FaaS), e serviços gerenciados. Fonte: Adaptado de (Shekhar, 2023).	20
Figura 4 – Funcionamento do padrão Circuit Breaker em sistemas distribuídos.	22
Figura 5 – Estrutura do Docker: cada contêiner encapsula uma aplicação, suas dependências e bibliotecas, todos isolados uns dos outros, rodando sobre o Docker Engine e compartilhando o mesmo sistema operacional do <i>host</i>	24
Figura 6 – Padrões fundamentais em arquitetura de microsserviços	26
Figura 7 – Arquitetura REST: comunicação stateless entre cliente e servidor, utilizando métodos HTTP (GET, POST, PUT, DELETE) para manipulação de recursos identificados por URI. O diagrama destaca os princípios REST, exemplos de URIs e resposta JSON, evidenciando a interface uniforme e a ausência de estado.	30
Figura 8 – Padrões de streaming no gRPC: (A) Unário, (B) Servidor, (C) Cliente, (D) Bidirecional.	31
Figura 9 – Arquitetura do Kubernetes: componentes principais, <i>control plane</i> , nós de trabalho, <i>Pods</i> e <i>containers</i>	34
Figura 10 – Arquitetura de observabilidade em ambientes Kubernetes, integrando métricas, logs e rastreamento distribuído.	36
Figura 11 – Relação entre a teoria de controle e a observabilidade em sistemas de software: a partir das saídas observáveis, busca-se reconstruir os estados internos do sistema, como latência, fila de mensagens e estado de sessão.	37
Figura 12 – Os três pilares da observabilidade: métricas, logs e rastreamento distribuído, cada um fornecendo uma perspectiva complementar para o monitoramento e diagnóstico de sistemas de microsserviços. . . .	39
Figura 13 – Fluxo de automação e resposta a incidentes: a partir da coleta de métricas, logs e rastreamentos, alertas são gerados e ações automáticas são executadas para mitigar falhas em ambientes de micros-serviços.	41
Figura 14 – Código de inicialização do servidor REST em <code>cmd/server/main.go</code> .	46
Figura 15 – Código de inicialização do gRPC em <code>cmd/grpc_server/main.go</code> . . .	49

Figura 16 – Dashboard do Grafana exibindo métricas de CPU e memória dos contêineres.	52
Figura 17 – Resultados da comparação entre REST/JSON e gRPC/Protobuf na transferência de 5.110 certificados.	55
Figura 18 – Resultados da comparação entre REST/JSON e gRPC/Protobuf na transferência de 15.110 certificados.	56
Figura 19 – Métricas de CPU e memória do servidor REST durante transferência de 15.110 certificados.	58
Figura 20 – Métricas de CPU e memória do servidor gRPC durante transferência de 15.110 certificados.	59
Figura 21 – Métricas de CPU e memória do servidor REST durante transferência de 5.110 certificados.	60
Figura 22 – Métricas de CPU e memória do servidor gRPC durante transferência de 5.110 certificados.	60

LISTA DE TABELAS

Tabela 1 – Comparação de protocolos de comunicação	25
Tabela 2 – Impacto da migração para microsserviços	27
Tabela 3 – Operações REST e códigos HTTP	29

LISTA DE ABREVIATURAS E SIGLAS

CPU	Central Processing Unit
GraphQL	Graph Query Language
gRPC	gRPC Remote Procedure Calls
HTTP	Hypertext Transfer Protocol
HTTP/2	Hypertext Transfer Protocol version 2
JSON	JavaScript Object Notation
REST	Representational State Transfer
URI	Uniform Resource Identifier

SUMÁRIO

1	INTRODUÇÃO	11
1.1	OBJETIVOS	13
1.1.1	Objetivo Geral	13
1.1.2	Objetivos Específicos	13
1.1.3	LIMITAÇÃO DO ESCOPO	13
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	PADRÕES ARQUITETURAIS	16
2.1.1	Arquitetura Monolítica	16
2.1.2	Event-Driven Architecture	18
2.1.3	Serverless	20
2.1.4	Microserviços	21
2.1.4.1	Docker e Microserviços	23
2.1.4.2	Mecanismos de Comunicação	25
2.1.4.3	Padrões Arquiteturais Essenciais	26
2.1.4.4	Desafios Operacionais e Soluções	26
2.1.4.5	Melhores Práticas de Implementação	27
2.1.4.6	Casos de Estudo Relevantes	27
2.1.4.7	Recomendações Estratégicas	27
2.1.5	Protocolos de Comunicação para Microserviços	28
2.1.5.1	REST (Representational State Transfer)	28
2.1.5.2	gRPC (gRPC Remote Procedure Calls)	30
2.2	OBSERVABILIDADE EM ARQUITETURAS DE MICROSERVIÇOS	32
2.2.1	Kubernetes	33
2.2.2	Relação com a Teoria de Controle	36
2.2.3	Pilares da Observabilidade	38
2.2.4	Automação e Resposta a Incidentes	40
3	DESENVOLVIMENTO	43
3.1	VISÃO GERAL DO PROJETO	43
3.2	AMBIENTE DE DESENVOLVIMENTO	43
3.3	ESTRUTURA DO PROJETO	44
3.4	IMPLEMENTAÇÃO DOS SERVIDORES	44
3.4.1	Servidor REST	44
3.4.2	Servidor gRPC	47
3.5	LÓGICA DE NEGÓCIO COMPARTILHADA	49
3.6	INTEGRAÇÃO COM OBSERVABILIDADE	51
3.7	METODOLOGIA DE COMPARAÇÃO	53
3.8	CONTAINERIZAÇÃO E ORQUESTRAÇÃO	53

4	RESULTADOS E ANÁLISE	54
4.1	METODOLOGIA DE EXECUÇÃO DOS TESTES	54
4.2	RESULTADOS OBTIDOS	55
4.2.1	Análise do Tamanho de Payload	56
4.2.2	Análise do Tempo de Transferência	57
4.3	ANÁLISE DE CONSUMO DE RECURSOS	57
4.3.1	Resultados do Teste com 15.110 Certificados	57
4.3.2	Resultados do Teste com 5.110 Certificados	59
5	CONSIDERAÇÕES FINAIS	62
	REFERÊNCIAS	64

1 INTRODUÇÃO

Em poucos anos, a complexidade dos sistemas de software em ascensão e a necessidade de escalabilidade; flexibilidade e pulseira continua têm consequentemente impulsionado a propagação da arquitetura de microsserviços ao invés da tradicional arquitetura monolítica (NIAZI et al., 2020).

Por que a fragmentação de aplicações de serviços independentes, torna as aplicações mais flexíveis em termos de desenvolvimento, teste, implantação e expansão a se harmonizar mais rapidamente e de modo mais eficiente, tornando-se apto para times distribuídos com ciclos de entrega mais curtos (NIAZI et al., 2020).

A arquitetura de microsserviços apresenta diversas vantagens: a modularização em serviços independentes facilita a manutenção e evolução de componentes isolados (JAMSHIDI, 2016); a escalabilidade granular permite o dimensionamento seletivo de cada serviço conforme demanda, esta arquitetura se alinha perfeitamente com ambiente *cloud*, provendo escalabilidade, flexibilidade e resiliência (Gaurav Shekhar, 2023).

O ciclo de desenvolvimento é acelerado, pois equipes menores podem trabalhar em paralelo em serviços distintos sem necessariamente conhecer os outros serviços, ciclos de entrega mais rápidos pela habilidade de fazer *deploy updates* independente para cada *micro serviço* facilitando atualizações frequentes, além disso temos a questão da alta confiabilidade, falhas são isoladas, minimizando os impactos, por exemplo, um *crash* em um sistema de pagamentos não vai para outros componentes como a autenticação do usuário (Swapnil K Shevate, 2021).

Contudo, embora a adoção de microsserviços traga benefícios inegáveis, também impõe desafios significativos: A orquestração e o gerenciamento de múltiplos serviços demandam ferramentas como *Kubernetes*, elevando a sobrecarga operacional e a curva de aprendizado (JAMSHIDI, 2016); cada serviço necessita de uma infraestrutura adicional, como separação de banco de dados, ferramentas de monitoramento e *logging*. Além disso há uma dificuldade maior no *debugging* da aplicação por conta dos *logs* serem distribuídos o que dificulta o *error tracing* e a resolução (Gaurav Shekhar, 2023).

Além disso, estudos como o de Niswar et al. [6] avaliam o impacto da escolha de protocolos de comunicação (*REST*, *gRPC*, *GraphQL*) sobre o desempenho de microsserviços, destacando a importância de decisões arquitetônicas bem-informadas para garantir uma observabilidade eficaz. Complementarmente, o trabalho de SHA et al. [7] investiga como *logs* de clusters *Kubernetes* podem ser usados para automatizar a análise de falhas em testes, evidenciando o papel crítico que a observabilidade desempenha na manutenção da qualidade de sistemas distribuídos.

Os três pilares fundamentais da observabilidade, conforme destacado por (Chi-

namanagonda, 2022), fornecem visibilidade integral sobre o comportamento de sistemas distribuídos.

O monitoramento acompanha continuamente o desempenho e disponibilidade do sistema através de métricas como utilização de CPU, consumo de memória e tempos de resposta. Como observado no MZ Computing Journal, ferramentas modernas transformam essa prática em estratégia proativa, alertando equipes sobre anomalias antes que escalem e mantendo a estabilidade operacional (Chinamanagonda, 2022).

O *logging* registra eventos sistêmicos detalhados - incluindo erros, advertências e informações operacionais - criando um histórico auditável. Em arquiteturas de microsserviços, onde serviços independentes geram volumosos fluxos de dados, sistemas centralizados de *logging* tornam-se indispensáveis para agregar e correlacionar informações distribuídas, viabilizando diagnóstico ágil de falhas (Chinamanagonda, 2022).

Já o *tracing* mapeia o fluxo transacional através de múltiplos serviços, expondo dependências e padrões de comunicação. Em ambientes distribuídos, este pilar identifica gargalos de desempenho e pontos críticos de falha no ciclo das requisições, sendo particularmente vital em microsserviços onde transações transversais exigem visibilidade topológica (Chinamanagonda, 2022). Conforme evidenciado no estudo, essa tríade evoluiu de abordagem reativa para modelo estratégico contínuo, capacitando resolução acelerada de incidentes em sistemas complexos.

O rastreamento distribuído complementa os pilares da observabilidade ao reconstruir o fluxo completo das requisições através de múltiplos serviços. Esta técnica mapeia jornadas transacionais complexas, revelando dependências ocultas e gargalos de desempenho que métricas isoladas não conseguem detectar (Chinamanagonda, 2022). Através da criação de linhas do tempo detalhadas, o rastreamento permite identificar pontos de falha e latência em cadeias de microsserviços, mesmo em ambientes altamente distribuídos. A sinergia entre métricas contínuas, registros contextualizados e mapas transacionais proporciona uma visão holística do sistema, essencial para a operação eficiente de arquiteturas modernas em escala.

A observabilidade consolida-se assim como catalisadora essencial na maturidade operacional de microsserviços. Conforme evidenciado por (Chinamanagonda, 2022) e (Sha et al., 2023), sua implementação estratégica converte dados distribuídos em inteligência acionável, permitindo: (1) manutenção proativa da estabilidade sistêmica; (2) diagnóstico preciso de falhas transacionais; (3) otimização contínua de desempenho. Este trabalho investiga como tais capacidades podem ser amplificadas mediante integração de padrões arquiteturais inovadores e ferramentas especializadas, respondendo aos desafios de complexidade destacados por (Shekhar, 2023).

1.1 OBJETIVOS

1.1.1 Objetivo Geral

Comparar o desempenho dos protocolos de comunicação *REST* e *gRPC* em arquiteturas de microsserviços, utilizando métricas de observabilidade para avaliar latência, *throughput* e uso de recursos.

1.1.2 Objetivos Específicos

- Implementação de dois microsserviços equivalentes, cada um empregando um dos protocolos em análise;
- Instrumentação dos serviços para expor métricas em uma rota, coletadas pelo prometheus;
- Execução de testes de carga controlados;
- Coleta e análise de métricas de latência, *throughput* e recursos pelo prometheus;
- Comparação estatística dos resultados para identificar o protocolo de melhor desempenho em diferentes cenários de carga.

1.1.3 LIMITAÇÃO DO ESCOPO

Este trabalho limita-se à comparação do desempenho entre os protocolos de comunicação *REST* e *gRPC* em dois serviços equivalentes implementados em uma arquitetura de *microsserviços*. A análise será conduzida em ambiente de testes controlado, com foco em métricas de observabilidade como latência (P95 e P99), *throughput* e uso de recursos.

Não serão abordados aspectos relacionados à segurança, autenticação, autorização, persistência de dados ou integração com sistemas legados. Da mesma forma, não se pretende avaliar outros protocolos de comunicação, como *GraphQL* ou *SOAP*, nem realizar testes em ambientes de produção. O uso de ferramentas como o *Prometheus* será restrito à coleta e análise de métricas, não sendo objetivo do estudo compará-las com outras soluções de monitoramento.

2 FUNDAMENTAÇÃO TEÓRICA

A arquitetura de *software* é um dos pilares centrais no desenvolvimento de sistemas computacionais modernos. Ela define a estrutura organizacional de um sistema, estabelecendo diretrizes para a disposição dos seus componentes, suas interações e restrições, além de orientar decisões técnicas que impactam diretamente na escalabilidade, manutenção e evolução do *software* ao longo do tempo (Jamshidi et al., 2016).

De acordo com (Jamshidi et al., 2016), a arquitetura de *software* pode ser compreendida como o conjunto de estruturas necessárias para raciocinar sobre o sistema, que compreende elementos de *software*, as relações entre eles e as propriedades de ambos. Em outras palavras, a arquitetura de *software* não se limita apenas à divisão do sistema em módulos, mas também envolve a definição de padrões de comunicação, protocolos, mecanismos de segurança, estratégias de escalabilidade e diretrizes para integração entre componentes.

A definição de uma arquitetura adequada é fundamental para garantir que o sistema atenda aos requisitos funcionais e não funcionais, como desempenho, segurança, confiabilidade e facilidade de manutenção. Segundo (Niazi et al., 2020), a escolha da arquitetura influencia diretamente a capacidade do sistema de evoluir e se adaptar a novas demandas, sendo um fator determinante para o sucesso de projetos de *software* em ambientes dinâmicos e competitivos.

Historicamente, a arquitetura monolítica foi o modelo predominante no desenvolvimento de sistemas. Nesse paradigma, todas as funcionalidades do *software* são implementadas em um único bloco, compartilhando o mesmo ambiente de execução, banco de dados e ciclo de vida (Niazi et al., 2020). Essa abordagem, embora simples de implementar e gerenciar em projetos de menor escala, apresenta limitações significativas à medida que o sistema cresce, como dificuldades de manutenção, escalabilidade restrita e maior risco de falhas generalizadas.

Com o avanço das demandas por sistemas mais flexíveis, escaláveis e resilientes, especialmente em ambientes de computação em nuvem, emergiu a arquitetura de microsserviços como uma alternativa ao modelo monolítico (Jamshidi et al., 2016; Shekhar, 2023). Os microsserviços propõem a fragmentação do sistema em serviços independentes, cada um responsável por uma funcionalidade específica e comunicando-se por meio de interfaces bem definidas. Essa abordagem permite que equipes distintas desenvolvam, testem e implantem serviços de forma autônoma, promovendo ciclos de entrega mais curtos e maior adaptabilidade às mudanças.

Além disso, a arquitetura de microsserviços facilita a adoção de práticas modernas de *DevOps*, automação de *deploys* e escalabilidade granular, alinhando-se às necessidades de negócios que exigem respostas rápidas e contínuas às mudanças

do mercado (Shekhar, 2023). No entanto, essa evolução também traz novos desafios, como a necessidade de ferramentas avançadas para orquestração, monitoramento e observabilidade, além de uma curva de aprendizado mais acentuada para as equipes de desenvolvimento (Jamshidi et al., 2016).

A definição de padrões arquiteturais é essencial para garantir a organização, escalabilidade e manutenção de sistemas de software. Diversos padrões foram consolidados na literatura internacional, cada um com características, vantagens e limitações específicas (Jamshidi et al., 2016).

Um dos padrões mais tradicionais é a arquitetura monolítica, na qual todas as funcionalidades do sistema são implementadas em um único bloco de código, compartilhando o mesmo ambiente de execução e banco de dados. Embora seja simples de desenvolver e implantar, esse modelo apresenta dificuldades de escalabilidade e manutenção à medida que o sistema cresce. Como afirmam (Niazi et al., 2020):

“A arquitetura monolítica é fácil de desenvolver e fazer *deploy*, mas, à medida que a aplicação cresce, torna-se difícil de gerenciar, escalar e manter.”

Essa limitação faz com que, em projetos de maior porte, a arquitetura monolítica se torne menos atrativa, especialmente quando há necessidade de frequentes atualizações e escalabilidade do sistema.

A arquitetura em camadas é amplamente utilizada em sistemas corporativos. Nesse modelo, o sistema é dividido em diferentes camadas, como apresentação, lógica de negócio e acesso a dados, cada uma com responsabilidades bem definidas. Essa separação facilita a manutenção, a reutilização de componentes e a evolução do sistema, além de permitir que equipes distintas trabalhem de forma mais organizada (Jamshidi et al., 2016).

A *Service-Oriented Architecture (SOA)* propõe a construção de sistemas a partir de serviços independentes, que se comunicam por meio de protocolos padronizados, como *REST* ou *SOAP*. O *SOA* foi fundamental para a integração de sistemas heterogêneos e para a criação de soluções mais flexíveis, permitindo que diferentes aplicações compartilhem funcionalidades de forma segura e controlada. No entanto, a complexidade de orquestração e a dependência de barramentos de serviço podem tornar a implementação desse padrão mais desafiadora (Jamshidi et al., 2016).

Nos últimos anos, a arquitetura de microsserviços ganhou destaque, principalmente devido à sua capacidade de promover a independência entre equipes, a escalabilidade granular e a facilidade de adoção de práticas modernas de *DevOps*. Cada microsserviço é responsável por uma funcionalidade específica e pode ser desenvolvido, testado, implantado e escalado de forma autônoma. Essa flexibilidade, no entanto, exige um investimento maior em ferramentas de monitoramento, orquestração e observabilidade, além de demandar uma curva de aprendizado mais acentuada para as equipes de desenvolvimento (Jamshidi et al., 2016; Shekhar, 2023; Niazi et al.,

2020).

2.1 PADRÕES ARQUITETURAIS

2.1.1 Arquitetura Monolítica

A arquitetura monolítica caracteriza-se pela integração de todos os componentes funcionais do sistema - interface de usuário, lógica de negócio e acesso a dados - em uma única unidade de *deploy* e execução (Niazi et al., 2020). Nesse modelo, o acoplamento entre módulos é extremamente elevado, uma vez que compartilham o mesmo espaço de memória, base de dados e ciclo de vida. Conforme destacado por (Farhan et al., 2023), essa estrutura promove rapidez no desenvolvimento inicial devido à simplicidade de configuração e depuração, sendo ideal para protótipos ou sistemas com escopo limitado.

Entretanto, à medida que o sistema expande sua base de código e adquire maior complexidade, surgem desafios significativos que impactam diretamente a manutenção e a evolução da aplicação. Um dos principais entraves é a escalabilidade vertical obrigatória: a única estratégia viável consiste na replicação da aplicação inteira, mesmo quando apenas componentes específicos demandam mais recursos (Niazi et al., 2020). Essa limitação resulta em desperdício de recursos computacionais e aumento de custos operacionais, dificultando o crescimento sustentável do sistema.

Outro ponto crítico refere-se à rigidez tecnológica. A adoção de novas tecnologias ou versões de bibliotecas exige a atualização do monolito como um todo, o que gera *lock-in* tecnológico e limita a capacidade de inovação (Jamshidi et al., 2016). Esse cenário dificulta a experimentação e a incorporação de soluções modernas, tornando o sistema menos adaptável às mudanças do mercado e às demandas dos usuários.

Além disso, há o comprometimento da confiabilidade. Falhas em módulos secundários podem paralisar toda a aplicação, uma vez que todos os componentes compartilham o mesmo ambiente de execução. Testes de carga realizados por (Farhan et al., 2023) demonstram que a indisponibilidade de um único módulo pode resultar em *downtime* total, evidenciando a fragilidade desse modelo em relação à resiliência.

Por fim, destaca-se a complexidade evolutiva. Modificações em qualquer parte do sistema exigem reteste e reimplantação completos, ampliando os riscos e o *lead time* para a entrega de novas funcionalidades. Esse processo torna-se cada vez mais oneroso à medida que o sistema cresce, dificultando a manutenção e a evolução contínua da aplicação.

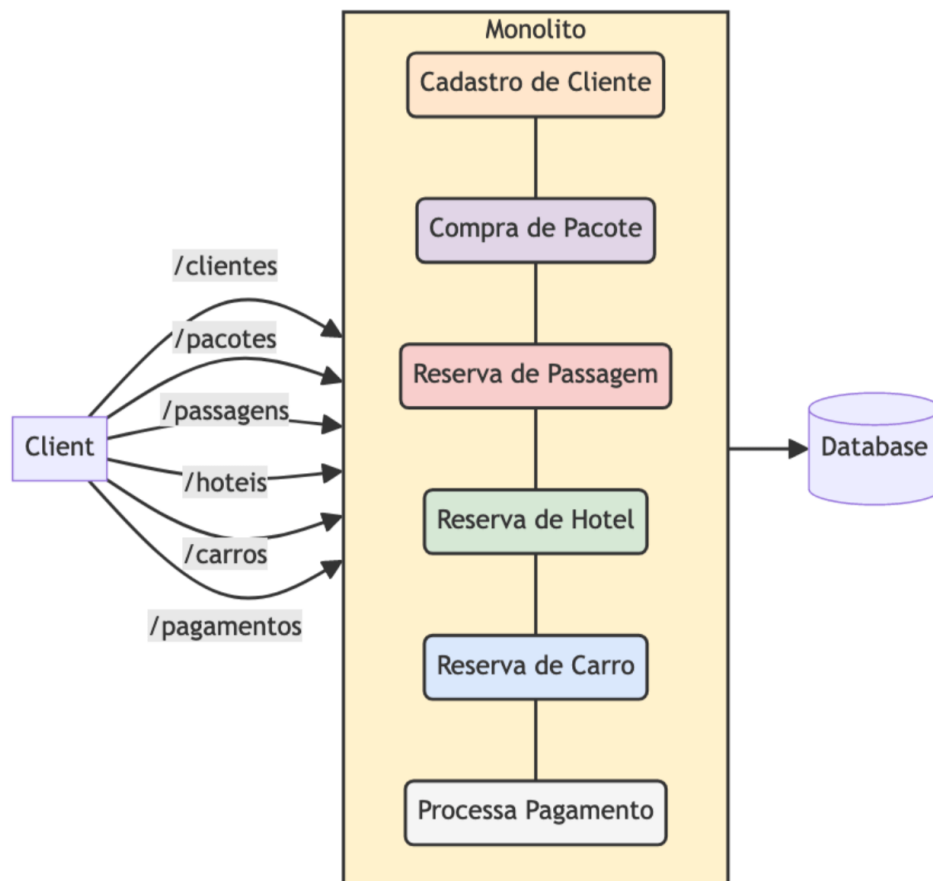


Figura 1 – Estrutura típica de arquitetura monolítica.

Como ilustrado na Figura 1, a arquitetura monolítica caracteriza-se pela centralização de todos os módulos e funcionalidades do sistema em uma única aplicação, onde as camadas como apresentação, lógica de negócio e acesso a dados encontram-se fortemente acopladas e compartilham o mesmo espaço de execução. O fluxo de requisições percorre camadas internamente acopladas, sem isolamento de processos. Essa centralização, embora simplifique o desenvolvimento inicial, apresenta limitações significativas em cenários de alta demanda e necessidade de escalabilidade.

Segundo Farhan et al. (Farhan et al., 2023), sistemas monolíticos apresentaram, em testes empíricos, 40% maior tempo de resposta sob carga crescente, 68% mais *downtime* durante atualizações e limite máximo de escalonamento em cinco instâncias, devido ao compartilhamento de recursos e à ausência de isolamento entre processos. Portanto, embora a arquitetura monolítica seja adequada para aplicações de menor porte ou com requisitos estáveis, suas restrições de escalabilidade, resiliência e flexibilidade tornam-se evidentes em ambientes dinâmicos e de grande escala, justificando a busca por alternativas arquiteturais mais modernas.

Portanto, embora a arquitetura monolítica seja adequada para aplicações de menor porte ou com requisitos estáveis, suas restrições de escalabilidade, resiliência e flexibilidade tornam-se evidentes em ambientes dinâmicos e de grande escala,

justificando a busca por alternativas arquiteturais mais modernas.

O fluxo de requisições percorre camadas internamente acopladas, sem isolamento de processos. Essa centralização explica os resultados empíricos de (Farhan et al., 2023), onde sistemas monolíticos apresentaram: O fluxo de requisições em sistemas monolíticos percorre camadas fortemente acopladas, sem qualquer tipo de isolamento entre processos. Essa centralização estrutural contribui para os resultados observados por (Farhan et al., 2023), que identificaram um aumento de 40% no tempo de resposta sob carga crescente, além de 68% mais *downtime* durante atualizações. Outro dado relevante é o limite máximo de escalonamento, que se restringe a cinco instâncias antes que ocorra degradação crítica do desempenho, evidenciando a dificuldade de adaptação a cenários de alta demanda.

Apesar dessas limitações, a arquitetura monolítica ainda mantém relevância em contextos específicos, como aplicações com tráfego previsível, equipes de desenvolvimento co-localizadas ou situações em que a simplicidade operacional é mais importante do que requisitos de elasticidade (Shekhar, 2023). No entanto, para sistemas empresariais modernos que exigem entrega contínua, resiliência e escalabilidade dinâmica, a viabilidade do modelo monolítico torna-se progressivamente reduzida, justificando a busca por alternativas arquiteturais mais flexíveis.

2.1.2 Event-Driven Architecture

A *Event-Driven Architecture* (EDA) é um paradigma arquitetural que se destaca pela capacidade de lidar com sistemas distribuídos, dinâmicos e de alta escalabilidade, sendo especialmente recomendada para aplicações que exigem resposta em tempo real e processamento assíncrono de grandes volumes de dados (Jamshidi et al., 2016). Nesse modelo, os componentes do sistema frequentemente implementados como microserviços comunicam-se por meio de eventos, que representam mudanças de estado ou ocorrências relevantes no domínio da aplicação. Cada evento é transmitido por um produtor e pode ser consumido por um ou mais consumidores, promovendo o desacoplamento entre os módulos e facilitando a evolução independente de cada serviço (Shekhar, 2023).

A adoção da EDA é comum em cenários como plataformas de *e-commerce*, sistemas financeiros, aplicações de *streaming* e ambientes de *Internet das Coisas* (IoT), onde a escalabilidade horizontal e a resiliência são requisitos críticos (Jamshidi et al., 2016; Shekhar, 2023). Nesses contextos, a arquitetura orientada a eventos permite que múltiplos serviços processem informações de forma paralela e reativa, reduzindo gargalos e aumentando a capacidade de resposta do sistema como um todo.

Segundo Jamshidi et al. (Jamshidi et al., 2016), a EDA contribui para a redução do acoplamento entre componentes, pois os serviços não dependem diretamente

uns dos outros, mas sim de eventos compartilhados por meio de um barramento ou intermediário, como um *message broker*. Essa característica facilita a manutenção, a escalabilidade e a resiliência, uma vez que falhas em um serviço não necessariamente impactam os demais, desde que o barramento de eventos permaneça operacional.

No entanto, a adoção da EDA traz desafios, como a complexidade na gestão do fluxo de eventos, a necessidade de garantir a ordem e a consistência das mensagens, além do aumento da dificuldade de depuração e testes em ambientes altamente distribuídos (Sha et al., 2023). Ainda assim, os benefícios em termos de escalabilidade, flexibilidade e resiliência tornam a arquitetura orientada a eventos uma escolha estratégica para sistemas modernos e de grande porte.

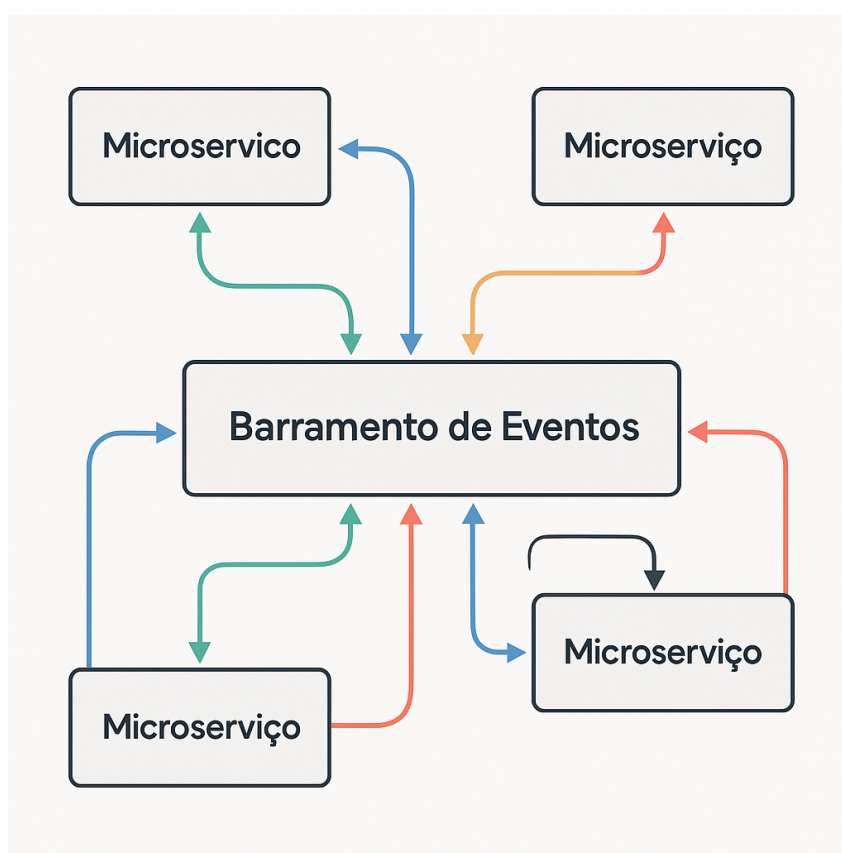


Figura 2 – Arquitetura Orientada a Eventos, ilustrando a comunicação assíncrona entre microserviços através de um barramento de eventos.

A Figura 2 apresenta um diagrama conceitual da Arquitetura Orientada a Eventos (*Event-Driven Architecture*). Nesta arquitetura, microserviços independentes comunicam-se de forma assíncrona através de um barramento de eventos central popularmente conhecido como *message broker*. Os microserviços podem tanto produzir eventos, enviando-os para o barramento, quanto consumir eventos, recebendo-os do barramento. Essa abordagem promove o desacoplamento entre os serviços, facilitando a escalabilidade, a resiliência e a manutenção do sistema.

A Figura 2 apresenta um diagrama conceitual da Arquitetura Orientada a Eventos (*Event-Driven Architecture*). Nesta arquitetura, microserviços independentes comunicam-se de forma assíncrona através de um barramento de eventos central popularmente

conhecido como *message broker*. Os microserviços podem tanto produzir eventos, enviando-os para o barramento, quanto consumir eventos, recebendo-os do barramento. Essa abordagem promove o desacoplamento entre os serviços, facilitando a escalabilidade, a resiliência e a manutenção do sistema.

2.1.3 Serverless

A arquitetura *serverless* tem se destacado com o avanço das plataformas de computação em nuvem, oferecendo uma abordagem inovadora para o desenvolvimento e a execução de aplicações. Nesse modelo, os desenvolvedores podem se concentrar exclusivamente na lógica de negócio, delegando a gestão da infraestrutura ao provedor de nuvem. Essa abstração permite maior agilidade no desenvolvimento e escalabilidade automática, embora apresente desafios relacionados à portabilidade e ao controle sobre o ambiente de execução (Shekhar, 2023).

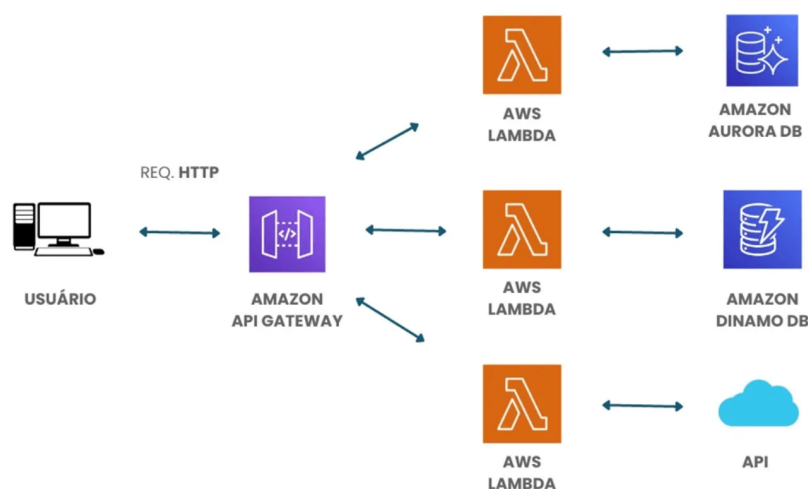


Figura 3 – Arquitetura Serverless típica, mostrando os componentes principais: *API Gateway*, Funções como Serviço (FaaS), e serviços gerenciados. Fonte: Adaptado de (Shekhar, 2023).

A Figura 3 ilustra um exemplo de arquitetura serverless, destacando seus componentes principais. O *API Gateway* atua como o ponto de entrada único para todas as requisições, roteando o tráfego para as funções de *backend*. As *Funções como Serviço (FaaS)*, como AWS Lambda ou Azure Functions, permitem a execução de código sob demanda, sem a necessidade de provisionar ou gerenciar servidores. Além disso, a arquitetura serverless utiliza *Serviços Gerenciados*, como bancos de dados e filas de mensagens, que são providos e mantidos pelo provedor de nuvem, simplificando ainda mais a operação e a manutenção da aplicação.

A adoção do padrão serverless elimina a necessidade de gerenciamento de servidores, permitindo que os desenvolvedores foquem exclusivamente na implementação

da lógica de negócios (Shekhar, 2023). A escolha do padrão arquitetural mais adequado depende de fatores como requisitos do sistema, contexto de negócio, equipe envolvida e recursos disponíveis. Compreender as vantagens e limitações de cada abordagem é essencial para alinhar a arquitetura às necessidades presentes e futuras do projeto (Jamshidi et al., 2016; Niazi et al., 2020).

2.1.4 Microsserviços

Como abordado anteriormente, a arquitetura de microsserviços decompõe sistemas em serviços autônomos. Sua implementação prática demanda atenção a três dimensões críticas: comunicação, observabilidade e orquestração. (Jamshidi et al., 2016; Niazi et al., 2020).

Entre os principais benefícios dos microsserviços está a escalabilidade granular, que permite dimensionar apenas os serviços que realmente demandam mais recursos, otimizando custos e desempenho (Shekhar, 2023). Além disso, a independência entre equipes é favorecida, pois times distintos podem desenvolver, testar e implantar serviços de forma autônoma, acelerando o ciclo de entrega e facilitando a adoção de práticas *DevOps* (Niazi et al., 2020; Farhan et al., 2023).

Outro ponto relevante é a resiliência: falhas em um serviço tendem a ser isoladas, reduzindo o impacto sobre o sistema como um todo (Farhan et al., 2023). Isso é especialmente importante em ambientes de missão crítica, onde a disponibilidade contínua é fundamental. Para garantir essa resiliência, padrões como o *Circuit Breaker* são frequentemente empregados.

A observabilidade em microsserviços demanda o uso de ferramentas especializadas para coleta e análise de métricas, logs e rastreamento distribuído (Madupati, 2023; Sha et al., 2023). Essa visibilidade é indispensável para identificar gargalos, diagnosticar falhas e garantir a saúde do sistema em produção. Soluções como Prometheus e Jaeger são amplamente utilizadas para monitorar o desempenho e rastrear o fluxo de requisições entre serviços, facilitando a manutenção e a evolução contínua da arquitetura.

Outro ponto relevante é a resiliência: falhas em um serviço tendem a ser isoladas, reduzindo o impacto sobre o sistema como um todo (Farhan et al., 2023). Isso é especialmente importante em ambientes de missão crítica, onde a disponibilidade contínua é fundamental. Para garantir essa resiliência, padrões como o *Circuit Breaker* são frequentemente empregados.

Por fim, a orquestração de microsserviços envolve o gerenciamento automatizado de implantação, escalonamento e atualização dos serviços, frequentemente utilizando plataformas como Kubernetes (Shekhar, 2023). Essa abordagem permite maior resiliência operacional, reduz o tempo de indisponibilidade e simplifica a administração de ambientes complexos. A automação dos processos de entrega contínua e

recuperação de falhas é um dos pilares para o sucesso de arquiteturas baseadas em microsserviços.

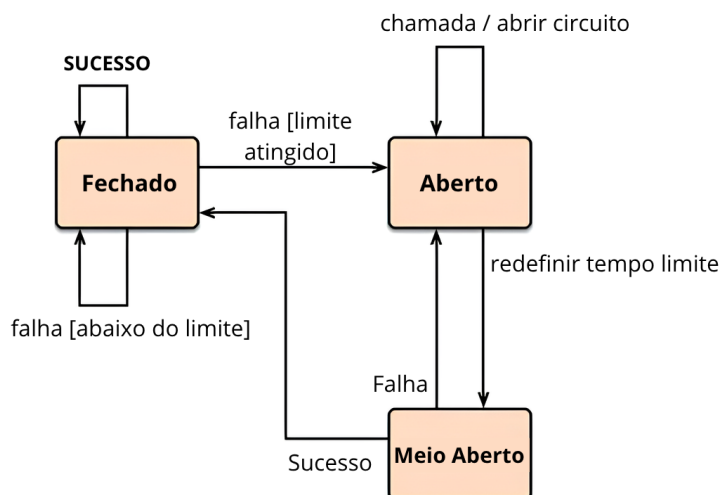


Figura 4 – Funcionamento do padrão Circuit Breaker em sistemas distribuídos.

O *Circuit Breaker* é um padrão arquitetural amplamente utilizado em sistemas distribuídos para aumentar a resiliência e evitar falhas em cascata (Nair et al., 2019; Fowler, 2012). Ele atua como um "interruptor" que monitora as chamadas a um serviço externo. Seu funcionamento é dividido em três estados principais: fechado, aberto e meio-aberto.

No estado fechado, todas as requisições são encaminhadas normalmente ao serviço. Se o número de falhas ultrapassar um limite configurado, o circuito entra no estado aberto, bloqueando novas tentativas e retornando imediatamente uma resposta de erro ou acionando um mecanismo de *fallback* uma solução alternativa, como dados em *cache* ou uma resposta padrão, para manter a aplicação funcionando de forma degradada. Após um tempo de espera, o circuito passa para o estado meio-aberto, permitindo algumas requisições de teste. Se essas requisições forem bem-sucedidas, o circuito retorna ao estado fechado; caso contrário, volta ao estado aberto, protegendo o sistema de sobrecarga e falhas sucessivas (Nair et al., 2019; Fowler, 2012).

Esse padrão é fundamental para garantir a robustez de sistemas baseados em microsserviços, pois impede que falhas em um serviço se propaguem e comprometam toda a aplicação. Estudos recentes destacam a importância do *Circuit Breaker* em arquiteturas resilientes, especialmente em ambientes de alta disponibilidade e missão crítica (Nair et al., 2019).

No entanto, a adoção de microsserviços também traz desafios consideráveis. A complexidade operacional aumenta, exigindo ferramentas avançadas para orquestração (como *Kubernetes*), monitoramento e observabilidade (Jamshidi et al., 2016;

Shekhar, 2023). Cada serviço pode demandar sua própria infraestrutura, banco de dados e mecanismos de autenticação, o que eleva a sobrecarga de gerenciamento (Niazi et al., 2020).

A observabilidade torna-se um aspecto central, pois a identificação e resolução de falhas em sistemas distribuídos requerem coleta e análise de métricas, *logs* e *traces* distribuídos (Madupati, 2023; Sha et al., 2023). Ferramentas como *Prometheus* e *Jaeger* são amplamente utilizadas para monitorar o desempenho e rastrear requisições entre serviços, facilitando o diagnóstico de problemas e a manutenção da qualidade do sistema (Ahmed et al., 2022).

Além disso, a escolha dos protocolos de comunicação entre microsserviços (*REST*, *gRPC*, *GraphQL*) impacta diretamente o desempenho, a flexibilidade e a observabilidade do sistema (Niswar et al., 2023). Estudos recentes destacam que decisões arquiteturais bem fundamentadas são essenciais para garantir a eficiência e a robustez de sistemas baseados em microsserviços (Niswar et al., 2023; Sha et al., 2023).

A arquitetura de microsserviços representa uma evolução paradigmática no desenvolvimento de software, decompondo sistemas complexos em serviços autônomos especializados. Conforme (Jamshidi et al., 2016), esta abordagem oferece:

- Autonomia tecnológica: Cada serviço pode utilizar linguagens e tecnologias distintas (*Java*, *Python*, *Node.js*)
- Resiliência aprimorada: Isolamento de falhas através de padrões como *Circuit Breaker*
- Entrega contínua: *Deploys* independentes permitindo múltiplas implantações diárias

2.1.4.1 Docker e Microsserviços

Docker revolucionou a forma como as aplicações são empacotadas, distribuídas e executadas, tornando-se um pilar fundamental para a adoção de microsserviços (Farhan et al., 2023). Ao encapsular cada serviço em um container, o Docker garante a portabilidade e a consistência do ambiente de execução, eliminando problemas de dependências e conflitos entre diferentes serviços.

A utilização de containeres Docker facilita a criação de ambientes isolados para cada microsserviço, permitindo que equipes de desenvolvimento trabalhem de forma independente e sem interferir no funcionamento de outros serviços (Farhan et al., 2023). Essa independência é crucial para acelerar o ciclo de entrega e promover a autonomia das equipes.

Além disso, o Docker simplifica o processo de implantação e escalonamento de microsserviços, pois os contêineres podem ser facilmente replicados e distribuídos em diferentes servidores ou ambientes de nuvem (Farhan et al., 2023). Essa flexibilidade

é essencial para garantir a escalabilidade e a resiliência das aplicações em produção.

A combinação de Docker e microsserviços também impulsiona a adoção de práticas *DevOps*, como a integração e a entrega contínua (*CI/CD*). Os contêineres Docker podem ser integrados em *pipelines* automatizados, permitindo que novas versões dos serviços sejam testadas e implantadas de forma rápida e segura (Farhan et al., 2023).

DevOps é uma abordagem cultural e técnica que busca integrar as áreas de desenvolvimento e operações por meio da automação de processos e da promoção da melhoria contínua. No contexto de arquiteturas baseadas em microsserviços, essa cultura se reflete na autonomia das equipes, que passam a ser responsáveis por todo o ciclo de vida dos seus serviços desde o desenvolvimento até a implantação, monitoramento e manutenção (Farhan et al., 2023).

CI/CD (Integração Contínua/Entrega Contínua) são práticas que automatizam o *pipeline* de desenvolvimento, desde a integração do código até a entrega em produção. A *CI* garante que as mudanças no código sejam integradas, testadas e validadas automaticamente, enquanto a *CD* automatiza a implantação dessas mudanças em ambientes de teste e produção, permitindo entregas mais rápidas e seguras (Farhan et al., 2023).

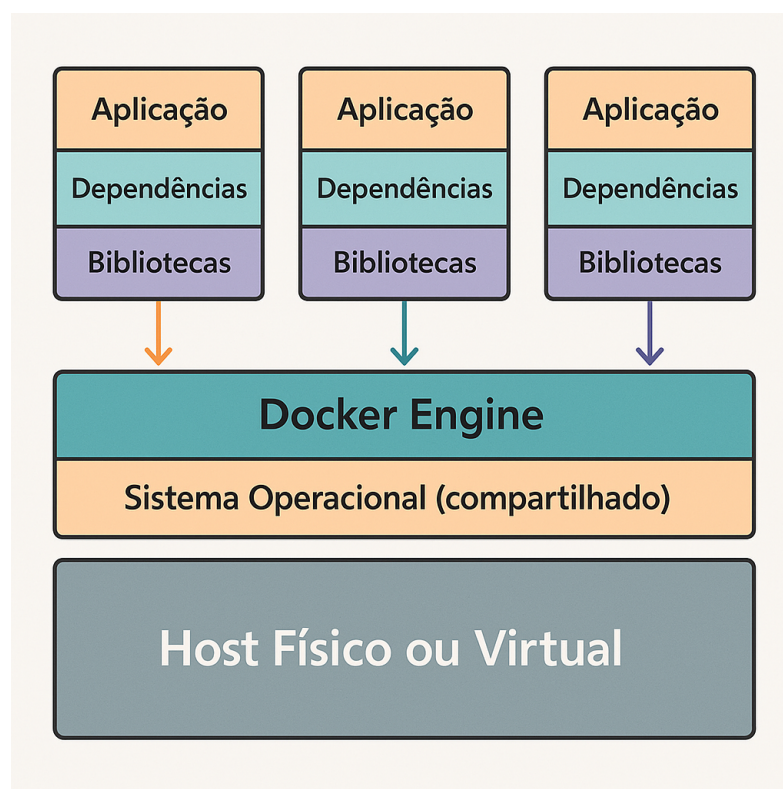


Figura 5 – Estrutura do Docker: cada contêiner encapsula uma aplicação, suas dependências e bibliotecas, todos isolados uns dos outros, rodando sobre o Docker Engine e compartilhando o mesmo sistema operacional do *host*.

A Figura 5 ilustra a arquitetura do Docker, mostrando como múltiplos contêineres podem ser executados de forma isolada sobre o mesmo *host* físico ou virtual.

Cada contêiner possui sua própria aplicação, dependências e bibliotecas, garantindo portabilidade e consistência, enquanto compartilham o sistema operacional subjacente por meio do Docker Engine.

Em resumo, o Docker desempenha um papel crucial na arquitetura de microserviços, fornecendo a base para a portabilidade, o isolamento, a escalabilidade e a automação das aplicações. Sua adoção é fundamental para garantir o sucesso e a eficiência de projetos baseados em microsserviços (Farhan et al., 2023).

2.1.4.2 Mecanismos de Comunicação

A comunicação entre serviços em arquiteturas de microsserviços pode ser realizada por diferentes mecanismos, cada um com características, vantagens e limitações específicas. A escolha do protocolo de comunicação é um fator determinante para o desempenho, a escalabilidade e a complexidade do sistema, influenciando diretamente a experiência do usuário e a eficiência operacional (Niswar et al., 2023; Maso, 2024).

Tabela 1 – Comparação de protocolos de comunicação

Protocolo	Latência	Complexidade	Casos de Uso
REST/HTTP	Alta	Baixa	Sistemas com baixa frequência
gRPC	Baixa	Média	Microserviços acoplados
Eventos (<i>Kafka</i>)	Assíncrona	Alta	Sistemas escaláveis

Conforme demonstrado na Tabela 1, a seleção do protocolo de comunicação deve considerar o perfil de uso e os requisitos de cada aplicação. O protocolo REST/HTTP é amplamente adotado devido à sua simplicidade e compatibilidade com diferentes plataformas, sendo indicado para cenários em que a frequência de requisições não é elevada e a interoperabilidade é prioridade (Fielding, 2000; Amazon Web Services, 2023). Por outro lado, o gRPC destaca-se pela baixa latência e eficiência na serialização de dados, tornando-se uma escolha adequada para integrações entre microsserviços que exigem alto desempenho e comunicação síncrona (Niswar et al., 2023; IBM, 2025).

Já os mecanismos baseados em eventos, como o uso de filas e *brokers* (exemplo: *Kafka*), possibilitam comunicação assíncrona e desacoplamento entre os serviços, favorecendo a escalabilidade horizontal e a resiliência do sistema (Jamshidi et al., 2016; Shekhar, 2023). Essa abordagem é especialmente recomendada para sistemas que processam grandes volumes de dados ou que demandam alta disponibilidade, pois permite que os serviços operem de forma independente, reduzindo o risco de falhas em cascata.

Além dos aspectos técnicos, a complexidade de implementação e manutenção também deve ser considerada. Protocolos como REST tendem a ser mais simples de configurar e depurar, enquanto soluções baseadas em eventos exigem infraestrutura

adicional e monitoramento especializado para garantir a integridade e a ordem das mensagens (Maso, 2024; Madupati, 2023). Dessa forma, a decisão sobre o mecanismo de comunicação deve ser pautada por uma análise criteriosa das necessidades do projeto, visando sempre o equilíbrio entre desempenho, escalabilidade e simplicidade operacional.

2.1.4.3 Padrões Arquiteturais Essenciais

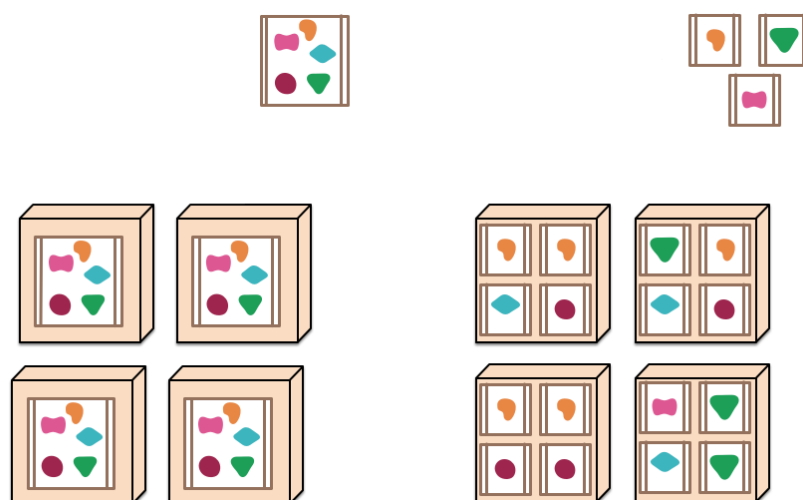


Figura 6 – Padrões fundamentais em arquitetura de microsserviços

A Figura 6 ilustra as diferenças fundamentais entre a arquitetura monolítica e a arquitetura de microsserviços. À esquerda, observa-se que uma aplicação monolítica agrupa todas as funcionalidades em um único processo. Para escalar esse modelo, é necessário replicar toda a aplicação em múltiplos servidores, o que pode gerar desperdício de recursos e limitações de flexibilidade.

À direita, a arquitetura de microsserviços distribui cada funcionalidade em processos independentes, permitindo que cada serviço seja desenvolvido, implantado e escalado separadamente. Isso proporciona maior eficiência no uso de recursos, facilita a manutenção e possibilita o crescimento gradual do sistema conforme a demanda por funcionalidades específicas.

Essa abordagem modular e distribuída é um dos pilares das arquiteturas modernas voltadas à escalabilidade, resiliência e agilidade no desenvolvimento.

2.1.4.4 Desafios Operacionais e Soluções

A implementação prática enfrenta obstáculos significativos:

- Observabilidade: Requer integração de métricas, *logs* e *traces* com ferramentas como:
 - *Prometheus* para monitoramento

- *Jaeger* para rastreamento distribuído
 - *EFK Stack* (Elasticsearch, Fluentd, Kibana) para análise
- Orquestração: Plataformas como *Kubernetes* gerenciam:
 - Escalonamento automático
 - Balanceamento de carga
 - Recuperação de falhas

2.1.4.5 Melhores Práticas de Implementação

Experiências de empresas líderes revelam estratégias eficazes:

- Amazon: Utiliza a prática do "time da pizza de dois" equipes pequenas e independentes, responsáveis por serviços autônomos do início ao fim.
- Netflix: Introduziu a cultura do *Chaos Engineering*, simulando falhas em produção para garantir a resiliência dos serviços.
- Spotify: Estruturou suas equipes em *squads* responsáveis por domínios funcionais, promovendo independência e entrega contínua.
- Uber: Implementou uma hierarquia de serviços baseada em criticidade, diferenciando serviços centrais de auxiliares para otimizar a operação.

2.1.4.6 Casos de Estudo Relevantes

Resultados empíricos demonstram impactos mensuráveis:

Tabela 2 – Impacto da migração para microsserviços

Métrica	Antes	Depois
Tempo de entrega	30 dias	2 dias
Disponibilidade	92%	99.95%
Custo de infraestrutura	\$100k/mês	\$65k/mês

Conforme dados na Tabela 2, organizações reportam melhorias significativas após transição bem-sucedida (Farhan et al., 2023; Shekhar, 2023).

2.1.4.7 Recomendações Estratégicas

A adoção deve considerar:

- Pré-requisitos: Maturidade em *DevOps* e cultura de automação
- Critérios: Complexidade sistêmica > 50k *LOC* > 15 desenvolvedores
- Alternativas: Arquiteturas híbridas para transição gradual
- Anti-padrões: Decomposição excessiva ("nanoserviços")

Em síntese, microsserviços oferecem vantagens transformacionais para sistemas complexos, mas exigem investimento proporcional em automação e governança (Niazi et al., 2020; Madupati, 2023).

2.1.5 Protocolos de Comunicação para Microsserviços

A seleção de protocolos de comunicação é um fator crítico em arquiteturas de microsserviços, influenciando diretamente o desempenho, acoplamento e capacidade de evolução do sistema. Dois padrões predominantes neste contexto são REST e gRPC, cada um com características técnicas distintas que os tornam adequados a cenários específicos (Niswar et al., 2023).

2.1.5.1 REST (Representational State Transfer)

O REST, formalizado por Fielding (Fielding, 2000) em sua tese seminal, é um estilo arquitetural baseado em princípios de recursos identificáveis por URI e operações padronizadas via verbos HTTP (GET, POST, PUT, DELETE). Sua adoção generalizada deve-se a:

- **Simplicidade:** Utiliza formatos legíveis como JSON, facilitando a depuração e a integração entre sistemas heterogêneos. No entanto, a serialização textual pode consumir de 30% a 50% mais banda em comparação com protocolos binários (Niswar et al., 2023).
- **Stateless:** Cada requisição carrega todo o contexto necessário, eliminando a necessidade de manter estado no servidor e favorecendo a escalabilidade horizontal (Fielding, 2000).
- **Interface Uniforme:** Princípios como a identificação única de recursos via URI e o uso de HATEOAS (*Hypermedia as the Engine of Application State*) garantem consistência e desacoplamento. Com HATEOAS, as respostas da API incluem hipermídia (links) que orientam o cliente sobre as próximas ações possíveis, promovendo navegação dinâmica e reduzindo o acoplamento entre cliente e servidor (Maso, 2024).
- **Compatibilidade:** Possui suporte nativo em navegadores e ampla adoção em APIs públicas, facilitando a integração com diferentes plataformas (Maso, 2024).

Limitações em cenários complexos:

- **Overhead de comunicação:** Serialização textual aumenta latência e consumo de CPU, especialmente em *payloads* grandes (Niswar et al., 2023).
- **Modelo síncrono:** Suporta apenas comunicação unária (request/response), inviabilizando fluxos assíncronos (Niswar et al., 2023).

- Falta de contratos rígidos: Validação de esquemas depende de implementações adicionais (Maso, 2024).

Operações Básicas e Códigos HTTP: A Tabela 3 sintetiza o mapeamento CRUD em REST, conforme padrões industriais (Fielding, 2000):

Tabela 3 – Operações REST e códigos HTTP

Método HTTP	Ação	Código Sucesso
GET	Ler recurso	200 (OK)
POST	Criar recurso	201 (Created)
PUT/PATCH	Atualizar recurso	200 (OK) ou 204 (No Content)
DELETE	Excluir recurso	204 (No Content)

Fonte: Adaptado de (Niswar et al., 2023) e (Fielding, 2000).

A adoção do REST em arquiteturas de microsserviços proporciona diversos benefícios, especialmente pela padronização da comunicação e pela facilidade de integração entre componentes heterogêneos. O uso de URIs para identificação de recursos, aliado à interface uniforme baseada em métodos HTTP, simplifica o desenvolvimento e a manutenção dos serviços, permitindo que diferentes equipes trabalhem de forma independente e escalável (Fielding, 2000; Maso, 2024).

Além disso, a independência de estado (*stateless*) facilita o balanceamento de carga e a replicação dos serviços, pois qualquer instância do servidor pode processar requisições sem depender de informações armazenadas em sessões. Essa característica é fundamental para ambientes distribuídos e de alta disponibilidade, comuns em sistemas modernos baseados em microsserviços (Fielding, 2000; Maso, 2024).

Por outro lado, é importante considerar que, apesar de sua simplicidade e ampla adoção, o REST pode apresentar limitações em cenários que exigem comunicação em tempo real, operações transacionais complexas ou validação rígida de contratos. Nesses casos, pode ser necessário complementar a arquitetura com outros padrões ou protocolos, como gRPC ou eventos assíncronos, para atender a requisitos específicos de desempenho e flexibilidade (Niswar et al., 2023).

Dessa forma, a arquitetura REST permanece como uma das principais escolhas para a construção de APIs robustas, escaláveis e de fácil integração, sendo amplamente utilizada tanto em aplicações web quanto em ambientes de microsserviços.

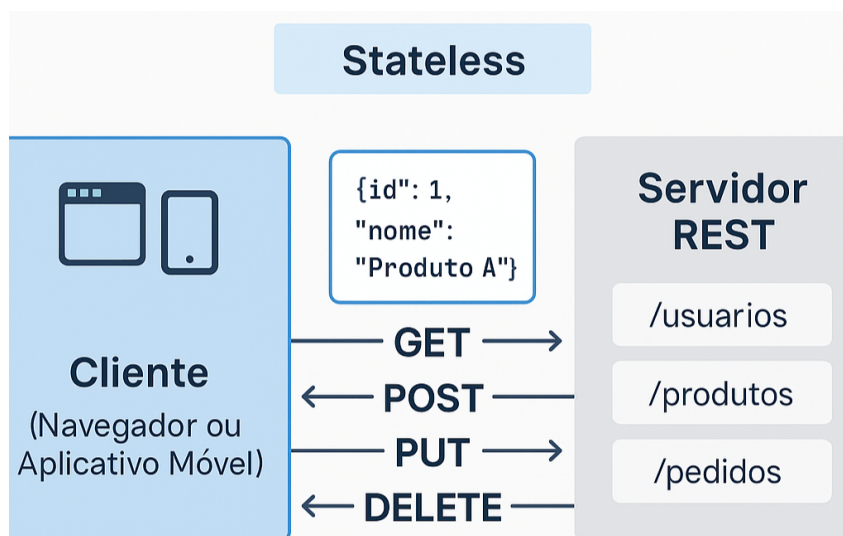


Figura 7 – Arquitetura REST: comunicação stateless entre cliente e servidor, utilizando métodos HTTP (GET, POST, PUT, DELETE) para manipulação de recursos identificados por URI. O diagrama destaca os princípios REST, exemplos de URIs e resposta JSON, evidenciando a interface uniforme e a ausência de estado.

A Figura 7 ilustra a arquitetura REST, evidenciando a troca de mensagens entre cliente e servidor por meio de métodos HTTP padronizados. Os recursos são identificados por URIs específicas, e as respostas são geralmente estruturadas em formato JSON. O princípio stateless garante que cada requisição seja independente, promovendo escalabilidade e simplicidade na integração entre sistemas.

2.1.5.2 gRPC (gRPC Remote Procedure Calls)

O gRPC é um *framework* aberto de alta performance para chamadas de procedimento remoto (RPC), desenvolvido pelo Google e lançado em 2015, que utiliza HTTP/2 e *Protocol Buffers* para comunicação binária eficiente (Niswar et al., 2023; Maso, 2024). Sua adoção em arquiteturas de microserviços deve-se a diferenciais como desempenho superior, suporte a streaming bidirecional, contratos fortemente tipados e geração automática de código.

- Desempenho superior: A serialização binária via *Protocol Buffers* reduz o tamanho dos *payloads* e a latência em relação a APIs REST tradicionais, conforme mensurações em ambientes controlados (Niswar et al., 2023).
- Streaming bidirecional: Suporte nativo a fluxos contínuos de dados (cliente-servidor, servidor-cliente e bidirecional), habilitando comunicação assíncrona para aplicações em tempo real (Maso, 2024).
- Contratos fortemente tipados: Definição explícita de serviços e estruturas de mensagens em arquivos `.proto`, garantindo compatibilidade e evolução controlada de APIs (Shekhar, 2023).

- Geração de código automática: Compilação de *stubs* clientes e servidores em múltiplas linguagens de programação a partir de arquivos `.proto`, promovendo consistência e redução de erros (Shekhar, 2023).

Apesar das vantagens, alguns desafios são observados:

- Acoplamento forte: Alterações nos contratos `.proto` exigem atualização sincronizada de clientes e servidores, aumentando a complexidade de evolução em sistemas distribuídos (Shekhar, 2023).
- Curva de aprendizagem: A definição de contratos e manipulação de fluxos de *streaming* pode ser complexa, especialmente para desenvolvedores familiarizados com paradigmas REST (Maso, 2024).

A Figura 8 apresenta os quatro padrões de comunicação suportados pelo gRPC, cada um adequado a diferentes cenários de integração entre serviços distribuídos.

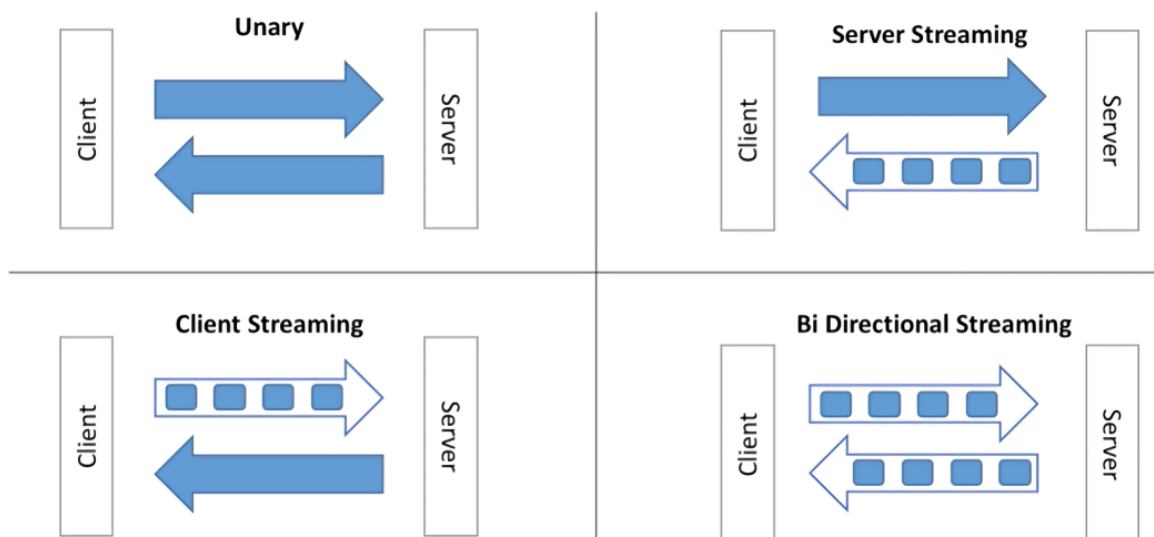


Figura 8 – Padrões de streaming no gRPC: (A) Unário, (B) Servidor, (C) Cliente, (D) Bidirecional.

No padrão *unary* (A), o cliente realiza uma chamada simples, enviando uma única requisição ao servidor e recebendo uma única resposta. Esse modelo é o mais próximo das operações tradicionais de APIs REST, sendo indicado para transações pontuais e de baixa complexidade, como consultas ou comandos diretos.

O padrão *server streaming* (B) ocorre quando o cliente envia uma requisição e o servidor responde com um fluxo contínuo de mensagens. Esse padrão é útil em situações em que o servidor precisa retornar uma grande quantidade de dados ou fornecer atualizações progressivas, como em relatórios extensos ou notificações em tempo real.

No padrão *client streaming* (C), o cliente envia um fluxo de mensagens ao servidor, que processa todas as informações recebidas e retorna uma única resposta ao final. Esse modelo é apropriado para cenários em que múltiplos dados precisam ser enviados de forma agrupada, como *uploads* de arquivos ou envio de lotes de registros.

Por fim, o padrão *bidirectional streaming* (D) permite que tanto o cliente quanto o servidor enviem múltiplas mensagens de forma independente e assíncrona, compartilhando um canal de comunicação contínuo. Esse padrão é especialmente poderoso para aplicações que exigem troca dinâmica de dados em tempo real, como *chats*, jogos *online* ou monitoramento de sensores, proporcionando máxima flexibilidade e desempenho na comunicação entre microserviços (Niswar et al., 2023; Shekhar, 2023).

Vantagens Técnicas Comprovadas: Estudos empíricos demonstram ganhos operacionais mensuráveis (Niswar et al., 2023):

- Redução de 35-45% no consumo de CPU durante serialização;
- Até 7x maior *throughput* em comunicações inter-serviços.

2.2 OBSERVABILIDADE EM ARQUITETURAS DE MICROSERVIÇOS

A migração de sistemas monolíticos para arquiteturas distribuídas baseadas em microserviços trouxe ganhos expressivos de escalabilidade e agilidade (Jamshidi et al., 2016). No entanto, essa transição introduziu um *gap* de visibilidade operacional que as técnicas tradicionais de monitoramento não conseguem preencher (Niazi et al., 2020).

Surge então o conceito de observabilidade, definido como a capacidade de inferir o estado interno de um sistema a partir de suas saídas externas (Madupati, 2023). Para organizações que operam centenas de serviços em ambientes Kubernetes, a observabilidade tornou-se um requisito fundamental (Shekhar, 2023).

Essa necessidade se justifica para manter SLOs (*Service Level Objectives*), que são metas específicas de desempenho e disponibilidade que um serviço deve atingir, garantindo a qualidade da experiência do usuário, reduzir *downtime* e acelerar ciclos de *deploy* (Shekhar, 2023). A complexidade inerente às arquiteturas de microserviços exige uma abordagem de monitoramento que vá além das métricas tradicionais (Jamshidi et al., 2016).

A observabilidade surge como uma solução para essa lacuna, permitindo que as equipes compreendam o comportamento do sistema em tempo real (Madupati, 2023). Isso possibilita a identificação proativa de problemas e a tomada de decisões informadas para otimizar o desempenho e a confiabilidade (Shekhar, 2023).

Diferentemente do monitoramento tradicional, que se concentra em métricas pré-definidas, a observabilidade permite explorar o sistema de forma dinâmica (Madupati, 2023). Essa exploração possibilita investigar o que não foi previsto inicialmente, crucial em ambientes de microserviços com comportamentos emergentes (Niazi et al., 2020).

A capacidade de inferir o estado interno a partir das saídas externas é fundamental para a resiliência e escalabilidade (Shekhar, 2023). Coletar e analisar dados de telemetria, como métricas, logs e traces, oferece uma visão holística do sistema

(Madupati, 2023).

Essa visão permite identificar gargalos de desempenho, erros e outros problemas que podem afetar a experiência do usuário (Ahmed et al., 2022). Além disso, a observabilidade possibilita monitorar o impacto das mudanças no código e na infraestrutura (Shekhar, 2023).

A adoção da observabilidade não é apenas uma questão de tecnologia, mas também de cultura e processos (Shekhar, 2023). As equipes precisam adotar uma mentalidade de responsabilidade compartilhada, onde os desenvolvedores monitoram e mantêm os serviços que criam (Madupati, 2023).

2.2.1 Kubernetes

Kubernetes é uma plataforma *open source* para orquestração de contêineres, amplamente utilizada em arquiteturas de microsserviços devido à sua capacidade de automatizar o gerenciamento, a implantação e o escalonamento de aplicações distribuídas (Shekhar, 2023). Sua adoção permite que empresas operem ambientes altamente dinâmicos, com múltiplos serviços sendo criados, atualizados e removidos de forma contínua.

A arquitetura do Kubernetes é composta por dois grandes blocos: o *Control Plane* e os nós de trabalho (*workers*). O *Control Plane* é responsável por gerenciar o estado desejado do *cluster*, tomando decisões sobre agendamento, monitoramento e controle dos recursos (Shekhar, 2023).

A crescente adoção de microsserviços impulsionou a necessidade de soluções de orquestração de contêineres que pudessem lidar com a complexidade do gerenciamento de aplicações distribuídas em larga escala (Jamshidi et al., 2016). O Kubernetes surgiu como uma resposta a essa demanda, oferecendo uma plataforma completa para automatizar o deploy, o escalonamento e a operação de contêineres em ambientes de produção.

Um dos principais diferenciais do Kubernetes é a sua capacidade de abstrair a infraestrutura subjacente, permitindo que as equipes de desenvolvimento se concentrem na criação de aplicações inovadoras, sem se preocupar com os detalhes da configuração e do gerenciamento dos servidores (Shekhar, 2023). Essa abstração simplifica o processo de deploy e reduz o tempo de lançamento de novas funcionalidades, impulsionando a agilidade e a competitividade das empresas.

Além disso, o Kubernetes oferece recursos avançados de auto-escalonamento, auto-recuperação e balanceamento de carga, garantindo que as aplicações permaneçam disponíveis e responsivas mesmo diante de picos de tráfego ou falhas de hardware (Shekhar, 2023). Essa resiliência é fundamental para garantir a satisfação dos usuários e a continuidade dos negócios em ambientes de alta demanda.

No topo da Figura 9, a interface do usuário representa as formas de interação

com o Kubernetes, seja por meio de interfaces gráficas (UI), linha de comando (CLI) ou a ferramenta *kubectl*, que permite executar comandos diretamente no *cluster*.

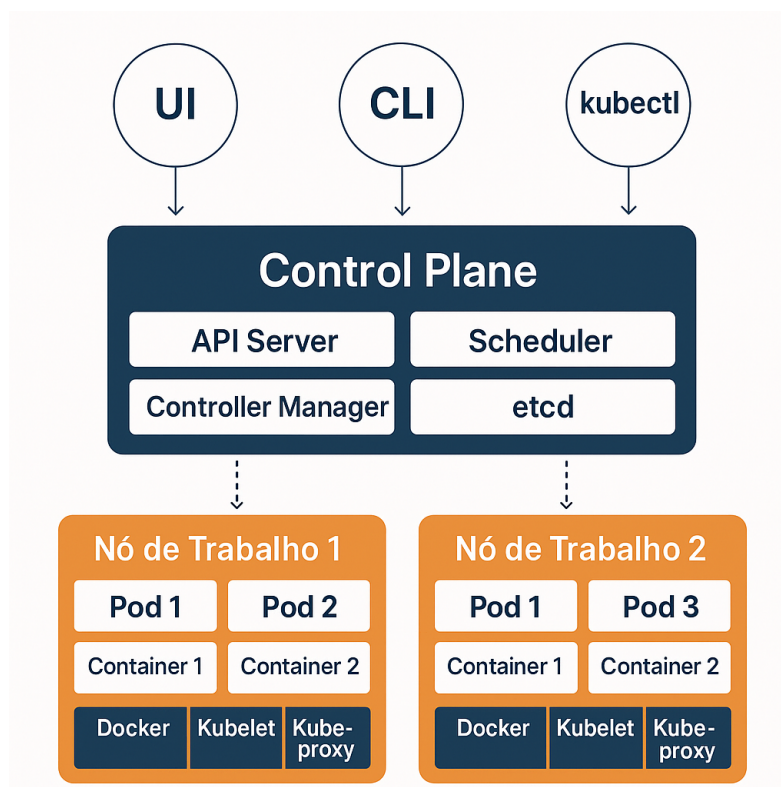


Figura 9 – Arquitetura do Kubernetes: componentes principais, *control plane*, nós de trabalho, *pods* e *containers*.

O *control plane* centraliza os principais componentes de controle do Kubernetes. O *API Server* é o ponto de entrada para todas as requisições, servindo como interface entre usuários, automações e o *cluster*. O *Scheduler* é responsável por decidir em qual nó os *pods* serão alocados, considerando recursos disponíveis e restrições. O *Controller Manager* gerencia os controladores que garantem que o estado real do *cluster* esteja sempre alinhado ao estado desejado. O *etcd* é o banco de dados chave-valor distribuído que armazena todas as informações de configuração e estado do *cluster* (Shekhar, 2023).

Nas laterais da figura estão os nós de trabalho (Nó de Trabalho 1 e Nó de Trabalho 2), que são servidores físicos ou virtuais responsáveis por executar as aplicações. Cada nó contém múltiplos *pods*, que são as menores unidades de implantação do Kubernetes. Cada *pod* pode conter um ou mais *containers*, que executam as aplicações e compartilham recursos de rede e armazenamento (Shekhar, 2023).

Na base de cada nó, encontram-se três componentes essenciais: o *Docker*, responsável por criar e gerenciar os *containers* o *Kubelet*, agente que garante que os *containers* estejam rodando conforme especificado pelo *control plane* e o *Kube-proxy*, que gerencia a comunicação de rede dentro e fora do nó, garantindo o roteamento correto das requisições (Shekhar, 2023).

As setas pontilhadas na figura indicam a comunicação entre o *control plane* e os nós de trabalho, evidenciando como o gerenciamento centralizado do *cluster* é realizado. Essa arquitetura modular e distribuída permite ao Kubernetes oferecer alta disponibilidade, escalabilidade e resiliência, características essenciais para ambientes de microsserviços modernos.

A observabilidade em Kubernetes é desafiadora, pois o ciclo de vida dos *pods* é efêmero e os serviços podem ser realocados entre nós do *cluster* a qualquer momento (Ahmed et al., 2022). Isso exige soluções que acompanhem essas mudanças em tempo real, garantindo visibilidade sobre o estado de cada componente do sistema.

Ferramentas como Prometheus, Grafana e Jaeger são essenciais nesse cenário. O Prometheus coleta métricas detalhadas de recursos, *pods* e serviços, enquanto o Grafana permite a visualização dessas métricas em *dashboards* customizáveis (Ahmed et al., 2022). O Jaeger, por sua vez, realiza o rastreamento distribuído, possibilitando a análise do caminho das requisições entre diferentes microsserviços.

Além dessas ferramentas, o Kubernetes integra nativamente eventos e logs que podem ser coletados por soluções como Fluentd e Elasticsearch, facilitando a centralização e análise dos dados (Ahmed et al., 2022). Essa integração é fundamental para detectar anomalias, identificar falhas e otimizar o uso de recursos do *cluster*.

Outro ponto relevante é o uso de *service mesh*, como Istio, que adiciona camadas de observabilidade sem necessidade de modificar o código das aplicações. O *service mesh* coleta métricas, logs e traces automaticamente, aumentando a granularidade e a precisão das informações (Kim; Lee, 2021).

A observabilidade em Kubernetes também permite a implementação de alertas inteligentes e automação de respostas a incidentes. Isso reduz o tempo de detecção e resolução de problemas, contribuindo para a alta disponibilidade e confiabilidade dos serviços (Ahmed et al., 2022).

A integração entre Kubernetes e ferramentas de observabilidade permite que equipes de operações e desenvolvimento tenham acesso a dados em tempo real sobre o funcionamento do ambiente. Essa visibilidade é essencial para identificar rapidamente falhas, gargalos de desempenho e comportamentos anômalos, reduzindo o tempo de resposta a incidentes e melhorando a experiência do usuário final (Ahmed et al., 2022).

Além disso, a observabilidade facilita a análise de causa raiz em ambientes distribuídos, onde uma falha em um serviço pode impactar diversos outros componentes do sistema. O rastreamento distribuído, por exemplo, permite acompanhar o fluxo de uma requisição por múltiplos microsserviços, identificando exatamente onde ocorreu o problema e possibilitando uma resolução mais ágil (Madupati, 2023).

Por fim, a adoção de práticas avançadas de observabilidade em Kubernetes contribui para a evolução contínua das aplicações. Com dados detalhados sobre o com-

portamento do sistema, as equipes podem tomar decisões baseadas em evidências, promovendo melhorias constantes na arquitetura, no desempenho e na confiabilidade dos serviços (Shekhar, 2023).

OBSERVABILITY ARCHITECTURE

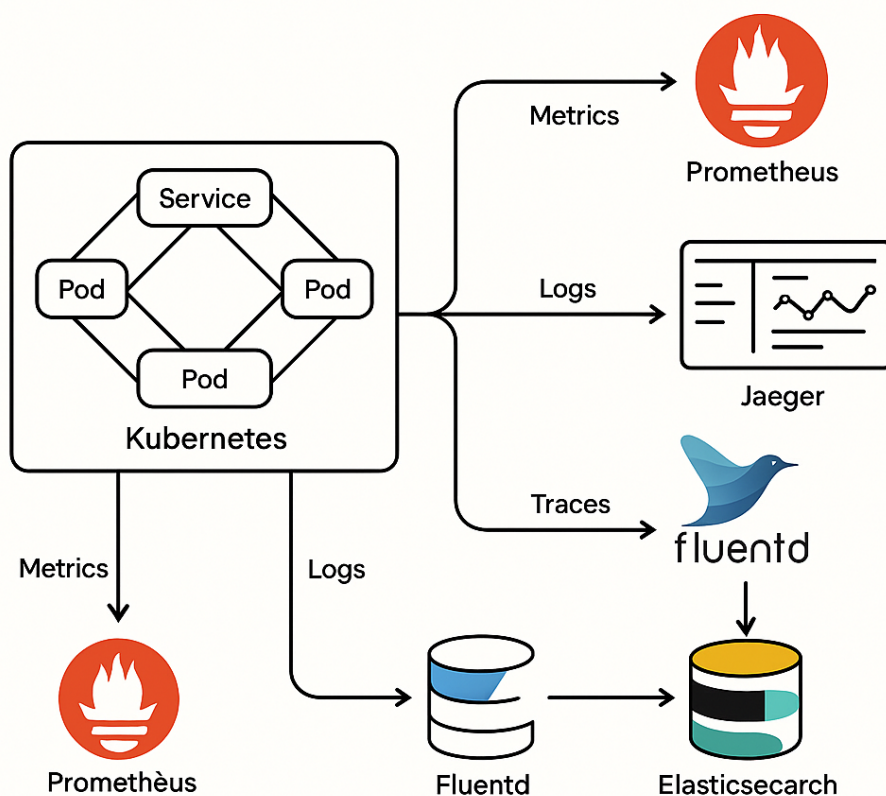


Figura 10 – Arquitetura de observabilidade em ambientes Kubernetes, integrando métricas, logs e rastreamento distribuído.

A Figura 10 ilustra como as principais ferramentas de observabilidade se integram ao Kubernetes, coletando dados de diferentes camadas do *cluster* e fornecendo uma visão unificada do ambiente.

Em resumo, a combinação de Kubernetes e práticas avançadas de observabilidade é indispensável para garantir a eficiência operacional, a escalabilidade e a resiliência de sistemas baseados em microsserviços (Shekhar, 2023; Ahmed et al., 2022).

2.2.2 Relação com a Teoria de Controle

Na teoria de sistemas lineares, um sistema é considerado observável quando, a partir das saídas disponíveis, é possível reconstruir completamente o vetor de estados internos do sistema ao longo do tempo (Madupati, 2023). Esse conceito, formalizado

por Kalman, é fundamental para o projeto de sistemas de controle robustos e confiáveis (Madupati, 2023).

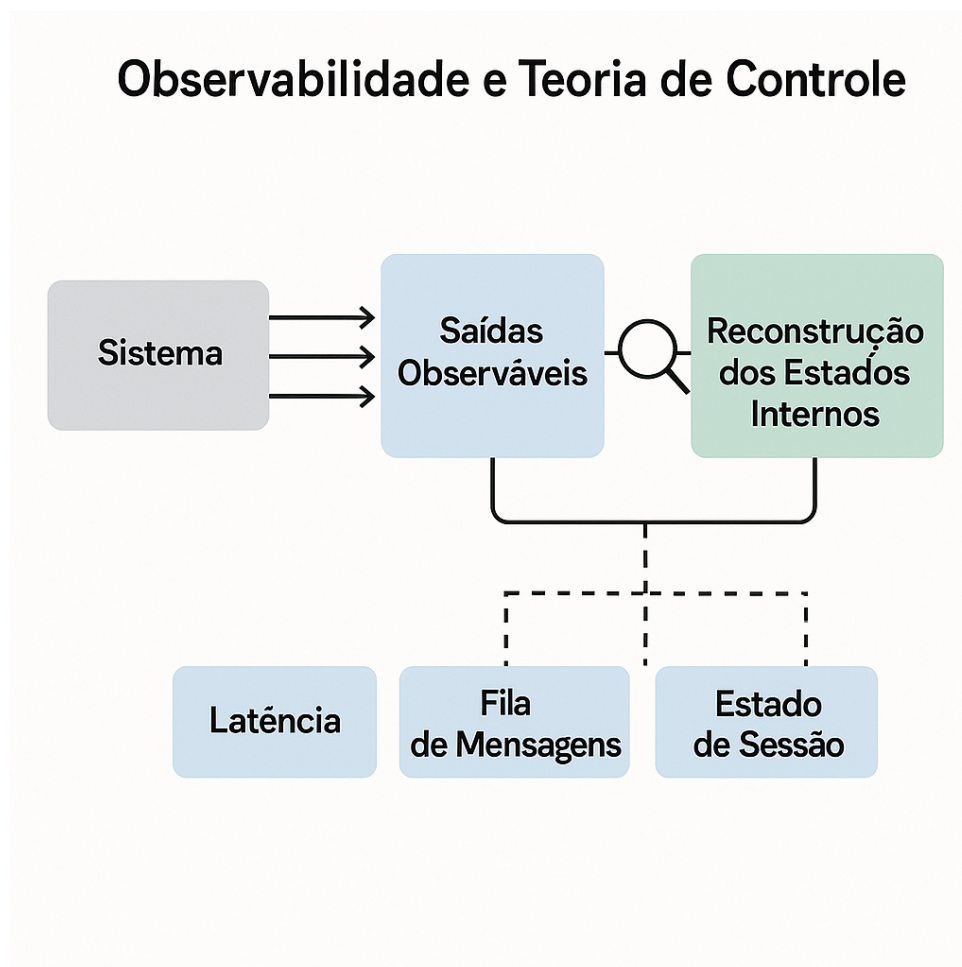


Figura 11 – Relação entre a teoria de controle e a observabilidade em sistemas de software: a partir das saídas observáveis, busca-se reconstruir os estados internos do sistema, como latência, fila de mensagens e estado de sessão.

A Figura 11 ilustra como, na teoria de controle, as saídas observáveis de um sistema são analisadas para reconstruir seus estados internos. No contexto de software, essa abordagem é aplicada para inferir variáveis como latência, fila de mensagens e estado de sessão a partir de sinais de telemetria coletados externamente (Madupati, 2023).

De forma análoga, em sistemas de software distribuídos, busca-se inferir variáveis internas, como latência de funções, tamanho de filas de mensagens e estado de sessões, a partir de sinais de telemetria coletados externamente (Madupati, 2023). Essa abordagem permite que equipes de operações e desenvolvimento compreendam o comportamento do sistema sem acesso direto ao seu interior (Madupati, 2023).

A perspectiva da observabilidade desloca o foco tradicional de “o que medir” para “como responder perguntas desconhecidas a priori” (Madupati, 2023). Ou seja, não basta definir métricas fixas; é necessário estruturar a coleta de dados de modo a

possibilitar investigações dinâmicas e flexíveis, respondendo a questões que surgem apenas diante de falhas ou comportamentos inesperados (Madupati, 2023).

Essa visão teórica fundamenta a importância de práticas avançadas de observabilidade em arquiteturas modernas, como microsserviços e ambientes Kubernetes (Madupati, 2023). Ao alinhar conceitos da engenharia de controle com a engenharia de software, reforça-se a necessidade de sistemas capazes de fornecer informações detalhadas e contextualizadas para a tomada de decisão rápida e precisa (Madupati, 2023).

2.2.3 Pilares da Observabilidade

A observabilidade em arquiteturas de microsserviços é sustentada por três pilares principais: métricas, logs e rastreamento distribuído (*tracing*) (Madupati, 2023). Cada um desses pilares fornece uma perspectiva única sobre o funcionamento do sistema, sendo essenciais para a detecção e resolução de problemas em ambientes distribuídos.

A implementação da observabilidade em arquiteturas de microsserviços traz diversos benefícios, como a detecção proativa de problemas, a redução do tempo de resolução de incidentes e a otimização do desempenho dos serviços (Madupati, 2023). No entanto, essa implementação também apresenta desafios significativos, como a complexidade da coleta e análise de dados em ambientes distribuídos e a necessidade de ferramentas e processos adequados.

Um dos principais desafios é a grande quantidade de dados gerados pelos microsserviços, que podem sobrecarregar os sistemas de monitoramento e dificultar a identificação de informações relevantes (Ahmed et al., 2022). Para lidar com esse volume de dados, é fundamental adotar técnicas de agregação, filtragem e amostragem, além de investir em ferramentas de análise escaláveis e eficientes.

Outro desafio importante é a necessidade de integrar diferentes fontes de dados, como métricas, logs e rastreamento distribuído, para obter uma visão holística do sistema (Madupati, 2023). Essa integração exige a padronização dos formatos de dados, a definição de modelos de correlação e a implementação de *dashboards* unificados, permitindo que as equipes compreendam o comportamento do sistema em tempo real e identifiquem rapidamente as causas de problemas.

A adoção dos pilares da observabilidade também impacta diretamente a cultura organizacional das equipes de desenvolvimento e operações (Shekhar, 2023). É necessário promover uma mentalidade de responsabilidade compartilhada, em que todos os envolvidos estejam atentos à saúde dos serviços e participem ativamente do processo de monitoramento e análise dos dados coletados.

Além disso, a escolha das ferramentas adequadas para cada pilar é um fator determinante para o sucesso da estratégia de observabilidade (Madupati, 2023). Solu-

ções como Prometheus para métricas, Fluentd para logs e Jaeger para rastreamento distribuído são amplamente utilizadas no ecossistema de microsserviços, pois oferecem integração nativa com plataformas como Kubernetes e facilitam a centralização das informações.

Por fim, a evolução constante das arquiteturas distribuídas exige que as práticas de observabilidade sejam revisadas e aprimoradas de forma contínua (Shekhar, 2023). Novos padrões, protocolos e ferramentas surgem regularmente, tornando fundamental que as equipes estejam atualizadas e preparadas para adaptar suas estratégias conforme as necessidades do ambiente e os avanços tecnológicos.

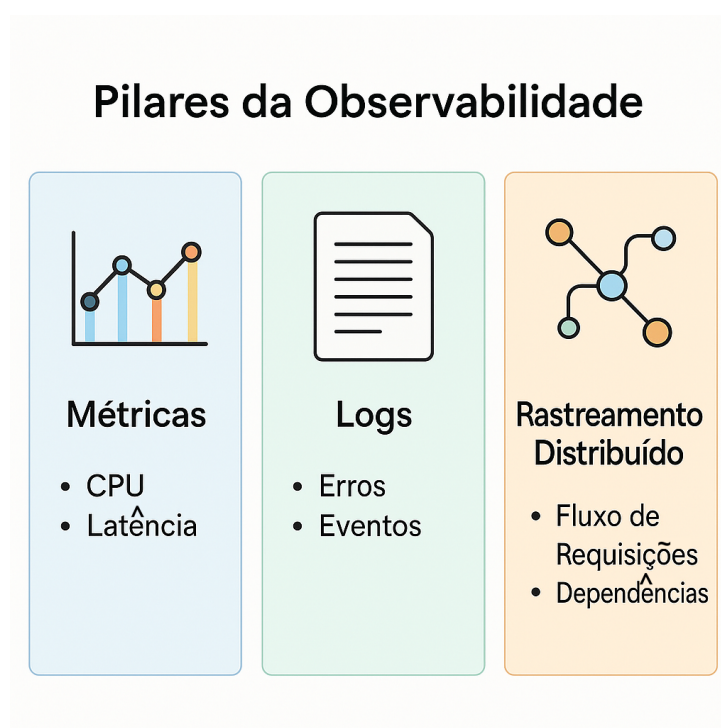


Figura 12 – Os três pilares da observabilidade: métricas, logs e rastreamento distribuído, cada um fornecendo uma perspectiva complementar para o monitoramento e diagnóstico de sistemas de microsserviços.

Fonte: ChatGPT (2025).

A Figura 12 ilustra os três pilares da observabilidade. Cada coluna representa um pilar fundamental: métricas, logs e rastreamento distribuído, com exemplos práticos de cada categoria.

As métricas são dados quantitativos coletados continuamente, como uso de CPU, memória, latência de requisições e taxa de erros (Ahmed et al., 2022). Elas permitem o monitoramento em tempo real do desempenho dos serviços, facilitando a identificação de tendências, gargalos e anomalias operacionais.

Os logs registram eventos detalhados que ocorrem durante a execução dos serviços, incluindo mensagens de erro, fluxos de execução e informações de auditoria (Madupati, 2023). A análise centralizada dos logs é fundamental para o diagnóstico de

falhas e para a compreensão do comportamento do sistema em situações específicas.

O rastreamento distribuído permite acompanhar o caminho de uma requisição ao longo de múltiplos microsserviços, identificando pontos de latência e dependências críticas (Madupati, 2023). Ferramentas como Jaeger e Zipkin são amplamente utilizadas para implementar esse pilar, proporcionando visibilidade de ponta a ponta em sistemas complexos.

A integração eficiente desses três pilares é indispensável para garantir uma observabilidade robusta, permitindo que equipes de desenvolvimento e operações respondam rapidamente a incidentes e promovam a melhoria contínua dos sistemas (Madupati, 2023).

2.2.4 Automação e Resposta a Incidentes

A evolução das práticas de observabilidade em microsserviços permitiu a integração de mecanismos de automação para resposta a incidentes, reduzindo o tempo de reação diante de falhas e melhorando a resiliência dos sistemas (Ahmed et al., 2022). Com a coleta contínua de métricas, logs e rastreamentos, é possível configurar alertas inteligentes que disparam ações automáticas, como reinício de serviços, escalonamento de recursos ou isolamento de componentes afetados.

A automação da resposta a incidentes depende fortemente da integração entre ferramentas de observabilidade e plataformas de orquestração, como Kubernetes (Shekhar, 2023). Por meio de regras e políticas definidas previamente, o sistema pode executar procedimentos corretivos sem intervenção humana, garantindo maior disponibilidade e minimizando o impacto de falhas sobre os usuários finais.

Além disso, a automação contribui para a padronização dos processos de resposta, reduzindo erros operacionais e promovendo uma cultura de melhoria contínua (Shekhar, 2023). A análise dos incidentes resolvidos automaticamente fornece insumos valiosos para o aprimoramento das estratégias de monitoramento e para a evolução das arquiteturas de microsserviços.

A adoção de automação na resposta a incidentes também facilita a implementação de testes e simulações de falhas, conhecidos como *chaos engineering* (Shekhar, 2023). Por meio dessas práticas, as equipes podem validar se as rotinas automatizadas realmente funcionam em cenários reais, identificando pontos de fragilidade e ajustando as políticas de resposta antes que ocorram incidentes em produção.

Outro ponto relevante é a rastreabilidade das ações automáticas executadas durante um incidente. O registro detalhado de cada etapa do processo, incluindo os alertas gerados, as decisões tomadas e as ações realizadas, é fundamental para auditoria, conformidade e aprendizado organizacional (Madupati, 2023). Essa rastreabilidade permite analisar a eficácia das respostas e aprimorar continuamente os fluxos de automação.

Por fim, a integração entre automação e resposta a incidentes com plataformas de comunicação, como sistemas de *chat* corporativo ou gerenciamento de tickets, potencializa a colaboração entre equipes (Ahmed et al., 2022). Notificações automáticas e relatórios detalhados garantem que todos os envolvidos estejam informados em tempo real, promovendo uma atuação coordenada e eficiente diante de eventos críticos.

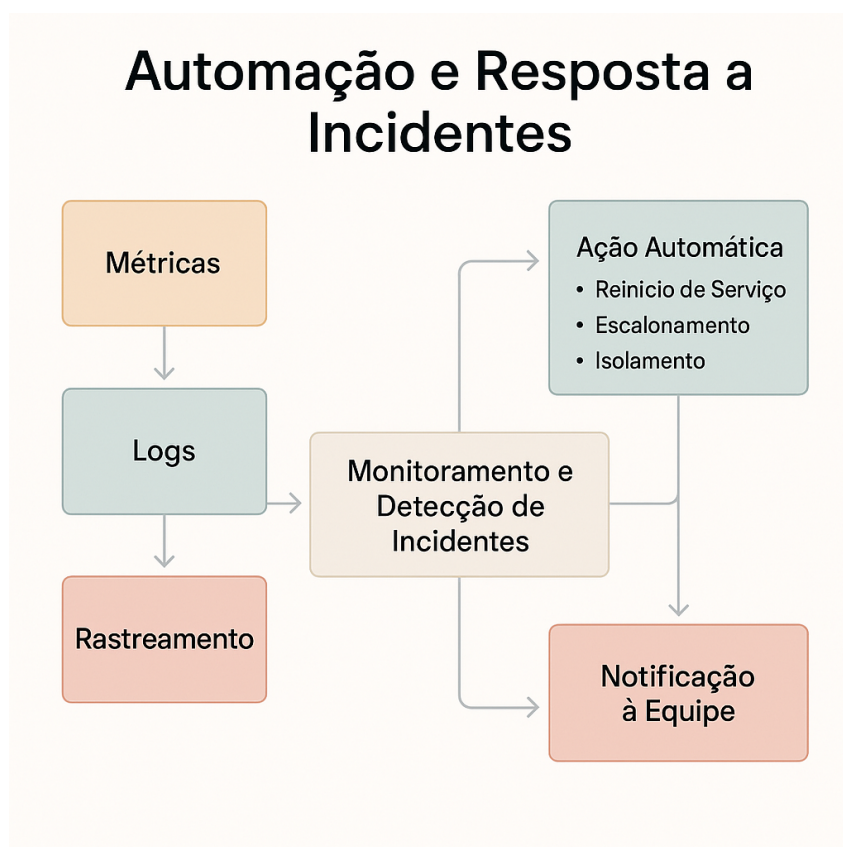


Figura 13 – Fluxo de automação e resposta a incidentes: a partir da coleta de métricas, logs e rastreamentos, alertas são gerados e ações automáticas são executadas para mitigar falhas em ambientes de microsserviços.

A Figura 13 ilustra o fluxo de automação e resposta a incidentes em ambientes de microsserviços. O processo inicia-se com a coleta de dados de telemetria, como métricas, logs e rastreamentos distribuídos. Em seguida, esses dados são analisados por sistemas de monitoramento, que geram alertas em caso de anomalias ou falhas detectadas. Por fim, rotinas automatizadas são acionadas para executar ações corretivas, como reiniciar serviços, escalar recursos ou isolar componentes problemáticos, garantindo a continuidade e a confiabilidade das aplicações (Ahmed et al., 2022).

A eficácia desse ciclo de automação depende da qualidade dos dados de telemetria coletados e da precisão dos algoritmos de detecção de anomalias (Madupati, 2023). É fundamental que as métricas, os logs e os rastreamentos forneçam informações detalhadas e relevantes sobre o comportamento dos serviços, permitindo que

os sistemas de monitoramento identifiquem rapidamente as causas de problemas e acionem as ações corretivas apropriadas.

3 DESENVOLVIMENTO

O tópico de desenvolvimento será tratado desde o ambiente utilizado até o funcionamento detalhado do sistema, abordando as tecnologias empregadas, a arquitetura adotada e as decisões de implementação.

3.1 VISÃO GERAL DO PROJETO

O projeto consiste em uma implementação prática para comparar os protocolos REST e gRPC em uma arquitetura de microsserviços. A aplicação foi desenvolvida em Go e possui dois servidores independentes: um utilizando REST com JSON e outro usando gRPC com Protocol Buffers. Ambos compartilham a mesma lógica de negócio para garantir uma comparação justa.

A funcionalidade implementada envolve a geração de certificados X.509. Essa escolha permite avaliar como cada protocolo se comporta ao lidar com diferentes volumes de dados e operações de serialização.

Para viabilizar a observabilidade, o projeto integra Prometheus para coleta de métricas, Grafana para visualização e cAdvisor para monitoramento dos contêineres Docker. Essas ferramentas permitem acompanhar métricas como latência, uso de CPU e memória, e tamanho de payload em tempo real.

3.2 AMBIENTE DE DESENVOLVIMENTO

O ambiente de desenvolvimento foi configurado utilizando Go versão 1.24.6 como linguagem principal. A escolha do Go se justifica por suas características de performance, suporte nativo a concorrência através de *goroutines* e amplo ecossistema para desenvolvimento de microsserviços. Além disso, a linguagem possui bibliotecas maduras tanto para construção de APIs REST quanto para implementação de serviços gRPC.

Para o desenvolvimento foi utilizado o *goland* como editor principal. O projeto utiliza Go *Modules* para gerenciamento de dependências, facilitando a reprodutibilidade do ambiente. As principais dependências incluem o framework gRPC (google.golang.org/grpc), Protocol Buffers (google.golang.org/protobuf), cliente Prometheus (github.com/prometheus/client_golang).

A containerização foi realizada através do *Docker*, permitindo que ambos os servidores executem em ambientes isolados e padronizados. O *Docker Compose* orquestra os serviços necessários (REST, gRPC, *Prometheus*, *Grafana* e *cAdvisor*), simplificando o processo de execução e testes.

3.3 ESTRUTURA DO PROJETO

A organização do código segue convenções da comunidade Go. O diretório `cmd/` contém os pontos de entrada das aplicações: `cmd/server/main.go` para o servidor REST e `cmd/grpc_server/main.go` para o servidor gRPC. Esses arquivos são responsáveis apenas pela inicialização dos servidores e configuração básica.

A lógica de negócio reside no diretório `internal/`, que está dividido em camadas. A pasta `internal/domain/` contém as estruturas de dados do domínio, como `Certificate` e `CertificateRequest`. A camada `internal/service/` implementa a lógica de geração de certificados X.509, sendo compartilhada por ambas as implementações de protocolo.

Para REST, os controladores HTTP estão em `internal/api/controller/`, enquanto o middleware de métricas Prometheus fica em `internal/api/middleware/`. Para gRPC, a implementação do servidor está em `internal/grpc/server.go`. As definições Protocol Buffer residem em `proto/certificate.proto`, e o código Go gerado automaticamente pelo compilador `protoc` fica na mesma pasta.

Os arquivos de configuração Docker ficam em `zarf/docker/`. E temos o diretório em `zarf/keys/` armazena as chaves RSA necessárias para assinatura de certificados, enquanto `zarf/certificates/` recebe os certificados gerados em tempo de execução.

3.4 IMPLEMENTAÇÃO DOS SERVIDORES

3.4.1 Servidor REST

O servidor REST foi implementado em `cmd/server/main.go` utilizando exclusivamente o pacote `net/http` da biblioteca padrão do Go, sem frameworks externos. O servidor escuta na porta 8000 e utiliza o multiplexador HTTP padrão para roteamento de requisições.

A inicialização do servidor segue uma estrutura de três camadas: serviços são instanciados primeiro (recebendo paths para chave privada e diretório de certificados), seguidos pelos controladores que dependem dos serviços, e finalmente as rotas HTTP são registradas. Esta arquitetura em camadas foi adotada para permitir reutilização da lógica de negócio entre o servidor REST e o servidor gRPC.

```
1 func main() {
2     // Initialize services
3     certificateService := service.NewCertificateService(
4         "zarf/keys/private_key.pem",
5         "zarf/certificates")
6
7     // Initialize controllers
8     certificateController := controller.NewCertificateController(
```

```

9      certificateService)
10
11     // Setup routes with Prometheus middleware
12     http.HandleFunc("/certificate/all",
13         middleware.PrometheusMiddleware(
14             certificateController.GetAllCertificates))
15     http.Handle("/metrics", promhttp.Handler())
16
17     log.Fatal(http.ListenAndServe(":8000", nil))
18 }

```

Código-Fonte 3.1 – Inicialização do servidor REST

Todos os *endpoints* foram envolvidos pelo `PrometheusMiddleware` para coleta automática de métricas. O *middleware* implementa três métricas principais: `http_requests_total` (contador de requisições por método, *endpoint* e *status*), `http_request_duration_seconds` (histograma de latência com *buckets* padrão do *Prometheus*), e `active_connections` (*gauge* de conexões ativas). Esta instrumentação foi fundamental para as análises de desempenho apresentadas no Capítulo 4.

```

1 func PrometheusMiddleware(next http.HandlerFunc) http.HandlerFunc {
2     return func(w http.ResponseWriter, r *http.Request) {
3         start := time.Now()
4         activeConnections.Inc()
5         defer activeConnections.Dec()
6
7         rw := &responseWriter{ResponseWriter: w, statusCode: http.
    ↪ StatusOK}
8         next(rw, r)
9
10        duration := time.Since(start).Seconds()
11        httpRequestDuration.WithLabelValues(r.Method, r.URL.Path).
    ↪ Observe(duration)
12        httpRequestsTotal.WithLabelValues(r.Method, r.URL.Path,
13            strconv.Itoa(rw.statusCode)).Inc()
14    }
15 }

```

Código-Fonte 3.2 – Middleware Prometheus implementado

```
    "ProjectTCC2/internal/api/controller"
    "ProjectTCC2/internal/api/middleware"
    "ProjectTCC2/internal/service"
)

var build = "develop" 2 usages & Matheus Coppi

func main() { & Matheus Coppi *
    fmt.Printf("Starting server with build: %s\n", build)

    // Initialize services
    certificateService := service.NewCertificateService("zarf/keys/private_key.pem", "outputDir: zarf/certificates")

    // Initialize controllers
    healthController := controller.NewHealthController(build)
    certificateController := controller.NewCertificateController(certificateService)
    stressController := controller.NewStressController()

    // Setup routes with middleware
    http.HandleFunc("/health", middleware.PrometheusMiddleware(healthController.Health))
    http.HandleFunc("/hello", middleware.PrometheusMiddleware(healthController.Hello))
    http.HandleFunc("/data", middleware.PrometheusMiddleware(func(w http.ResponseWriter, r *http.Request) {
        time.Sleep(100 * time.Millisecond)
        healthController.Data(w, r)
    }))
    http.HandleFunc("/stress", middleware.PrometheusMiddleware(stressController.Stress))
    http.HandleFunc("/certificate", middleware.PrometheusMiddleware(certificateController.GenerateCertificate))
    http.HandleFunc("/certificate/get", middleware.PrometheusMiddleware(certificateController.GetCertificate))
    http.HandleFunc("/certificate/list", middleware.PrometheusMiddleware(certificateController.ListCertificates))
    http.HandleFunc("/certificate/all", middleware.PrometheusMiddleware(certificateController.GetAllCertificates))
    http.HandleFunc("/certificate/benchmark", middleware.PrometheusMiddleware(certificateController.BenchmarkGetAll))
    http.Handle("/metrics", promhttp.Handler())

    fmt.Println("Server starting on :8080")
    log.Fatal(http.ListenAndServe(":" + "8080", nil))
}
```

Figura 14 – Código de inicialização do servidor REST em cmd/server/main.go.

Fonte: Elaborado pelo autor (2025).

Os endpoints implementados no servidor REST são:

- POST /certificate: Gera um novo certificado X.509 baseado nos parâmetros recebidos em JSON
- GET /certificate/get?filename=X: Recupera um certificado específico por nome de arquivo
- GET /certificate/list: Lista os nomes de todos os certificados armazenados
- GET /certificate/all: Retorna um mapa completo com todos os certificados e seus conteúdos em formato JSON (endpoint principal utilizado nos testes de comparação)
- GET /certificate/benchmark: Endpoint de medição que retorna estatísticas de tempo de busca, serialização e tamanho do payload JSON
- GET /metrics: Expõe métricas do Prometheus para scraping
- GET /health: Verifica o status de saúde do serviço

O *endpoint* /certificate/all foi escolhido como base de comparação por retornar o volume completo de dados, permitindo avaliar o comportamento dos protocolos sob carga de transferência significativa. A serialização de resposta é feita com `json.Marshal` da biblioteca padrão Go.

3.4.2 Servidor gRPC

O servidor gRPC foi implementado em `cmd/grpc_server/main.go` escutando na porta 50051. O contrato de serviço foi definido utilizando Protocol Buffers versão 3 no arquivo `proto/certificate.proto`.

```
1 syntax = "proto3";
2 package certificate;
3
4 service CertificateService {
5   rpc GetAllCertificates(GetAllRequest) returns (GetAllResponse);
6   rpc ListCertificates(ListRequest) returns (ListResponse);
7   rpc GetCertificate(GetRequest) returns (GetResponse);
8   rpc BenchmarkGetAll(GetAllRequest) returns (BenchmarkResponse);
9 }
10
11 message GetAllResponse {
12   map<string, string> certificates = 1;
13 }
14
15 message BenchmarkResponse {
16   int32 certificate_count = 1;
17   int64 payload_size_bytes = 2;
18   double payload_size_mb = 3;
19   int64 fetch_duration_ms = 4;
20   int64 serialize_duration_ms = 5;
21   int64 total_duration_ms = 6;
22   string serialization_format = 7;
23 }
```

Código-Fonte 3.3 – Definição do serviço em Protocol Buffers (`certificate.proto`)

O método `GetAllCertificates` é equivalente ao endpoint REST `/certificate`, retornando um mapa de certificados. O método `BenchmarkGetAll` fornece estatísticas detalhadas de serialização Protobuf para comparação com JSON.

O código Go foi gerado automaticamente a partir do arquivo `.proto` usando os comandos:

```
1 protoc --go_out=. --go_opt=paths=source_relative \
2       --go-grpc_out=. --go-grpc_opt=paths=source_relative \
3       proto/certificate.proto
```

A inicialização do servidor gRPC configurou `MaxRecvMsgSize` e `MaxSendMsgSize` para 50 MB, necessário porque a transferência de 15.110 certificados gera payloads próximos a 19 MB, superando o limite padrão de 4 MB do gRPC.

```
1 func main() {
2     certificateService := service.NewCertificateService(
3         "/zarf/keys/private_key.pem",
4         "/zarf/certificates")
5
6     maxMsgSize := 50 * 1024 * 1024 // 50MB
7     grpcServer := grpc.NewServer(
8         grpc.MaxRecvMsgSize(maxMsgSize),
9         grpc.MaxSendMsgSize(maxMsgSize),
10    )
11
12    certificateServer := grpcserver.NewCertificateServer(
13        certificateService)
14    pb.RegisterCertificateServiceServer(grpcServer, certificateServer
15    ↪ )
16
17    listener, err := net.Listen("tcp", ":50051")
18    grpcServer.Serve(listener)
19 }
```

Código-Fonte 3.4 – Inicialização do servidor gRPC

A implementação dos métodos RPC em `internal/grpc/server.go` delega a lógica de negócio para o mesmo `CertificateService` utilizado pelo servidor REST, garantindo comportamento idêntico entre os protocolos. A diferença reside exclusivamente na camada de transporte (HTTP/2 vs HTTP/1.1) e serialização (Protobuf vs JSON).

Cada método RPC foi implementado como uma função que recebe `context` e uma mensagem de requisição, retornando uma mensagem de resposta e erro. Por exemplo, `GetAllCertificates` chama `certificateService.GetAllCertificates()`, recebe o mapa de certificados, e o encapsula em `pb.GetAllResponse{Certificates: certificates}`. O framework gRPC automaticamente serializa essa resposta em Protobuf binário antes de transmitir pela rede.

O método `BenchmarkGetAll` implementa medições detalhadas similares ao *end-point* REST equivalente. Ele mede o tempo de busca dos certificados, serializa manualmente a resposta usando `proto.Marshal` para obter o tamanho exato do *payload Protobuf*, e retorna estatísticas incluindo contagem de certificados, tamanho em *bytes*, tempos de busca e serialização. Esta implementação foi essencial para comparar o

desempenho de serialização entre JSON e Protobuf de forma controlada.

O servidor gRPC foi containerizado usando Dockerfile multi-stage em `zarf/docker/grpc.dockerfile`. O primeiro estágio compila o código Go com `CGO_ENABLED=0` para gerar binário estático, e o segundo estágio copia o binário para imagem *Alpine Linux* mínima. Os volumes Docker mapeiam `zarf/keys` em modo read-only (fornecendo a chave privada RSA) e `zarf/certificates` em modo read-write (compartilhado com o servidor REST para armazenar certificados). Este compartilhamento de volumes garante que ambos os servidores acessem exatamente os mesmos arquivos de certificados durante os testes.

```

"google.golang.org/grpc"

grpcserver "ProjectTCC2/internal/grpc"
"ProjectTCC2/internal/service"
pb "ProjectTCC2/proto"
)

var build = "develop" 1 usage & Matheus Coppi

func main() { & Matheus Coppi
    fmt.Printf("Starting gRPC server with build: %s\n", build)

    certificateService := service.NewCertificateService( privateKeyPath: "/zarf/keys/private_key.pem", outputDir: "/zarf/certificates")

    maxMsgSize := 50 * 1024 * 1024 // 50MB
    grpcServer := grpc.NewServer(
        grpc.MaxRecvMsgSize(maxMsgSize),
        grpc.MaxSendMsgSize(maxMsgSize),
    )
    certificateServer := grpcserver.NewCertificateServer(certificateService)
    pb.RegisterCertificateServiceServer(grpcServer, certificateServer)

    listener, err := net.Listen(network: "tcp", address: ":50051")
    if err != nil { log.Fatalf("Failed to listen: %v", err) }

    fmt.Println(a... "gRPC server starting on :50051")
    fmt.Println(a... "Services:")
    fmt.Println(a... "  certificate.CertificateService")
    fmt.Println(a... "    - GetAllCertificates")
    fmt.Println(a... "    - ListCertificates")
    fmt.Println(a... "    - GetCertificate")
    fmt.Println(a... "    - BenchmarkGetAll")

    if err := grpcServer.Serve(listener); err != nil {
        log.Fatalf("Failed to serve: %v", err)
    }
}

```

Figura 15 – Código de inicialização do gRPC em `cmd/grpc_server/main.go`.

Fonte: Elaborado pelo autor (2025).

Os métodos RPC implementados em `internal/grpc/server.go` são:

- `GetAllCertificates`: Retorna todos os certificados como mapa (método principal usado nos testes)
- `ListCertificates`: Lista apenas os nomes dos arquivos de certificados
- `GetCertificate`: Recupera um certificado específico por nome
- `BenchmarkGetAll`: Retorna estatísticas de tempo de busca, serialização Protobuf e tamanho do payload binário

3.5 LÓGICA DE NEGÓCIO COMPARTILHADA

Ambos os servidores (REST e gRPC) compartilham a mesma implementação de lógica de negócio através do `CertificateService`. Esta decisão arquitetural foi

fundamental para garantir que a comparação avalie exclusivamente as diferenças de protocolo e serialização, não diferenças na lógica de negócio.

A interface do serviço define quatro operações:

```

1 type CertificateService interface {
2     GenerateCertificate(req domain.CertificateRequest) (*domain.
      ↪ Certificate, error)
3     GetCertificate(filename string) (string, error)
4     ListCertificates() ([]string, error)
5     GetAllCertificates() (map[string]string, error)
6 }

```

Código-Fonte 3.5 – Interface CertificateService

O método GetAllCertificates é o mais relevante para os testes de comparação, pois retorna todos os certificados armazenados:

```

1 func (s *certificateService) GetAllCertificates() (map[string]string,
      ↪ error) {
2     files, err := filepath.Glob(filepath.Join(s.outputDir, "
      ↪ certificate_*.crt"))
3     if err != nil {
4         return nil, fmt.Errorf("failed to list certificates: %w", err
      ↪ )
5     }
6
7     certificates := make(map[string]string)
8     for _, file := range files {
9         filename := filepath.Base(file)
10        certBytes, err := os.ReadFile(file)
11        if err != nil {
12            return nil, fmt.Errorf("failed to read certificate %s: %w
      ↪ ", filename, err)
13        }
14        certificates[filename] = string(certBytes)
15    }
16
17    return certificates, nil
18 }

```

Código-Fonte 3.6 – Implementação de GetAllCertificates

Este método busca todos os arquivos .crt no diretório configurado, lê cada arquivo e retorna um mapa onde a chave é o nome do arquivo e o valor é o conteúdo

do certificado em formato PEM. Este mapa é então serializado em JSON (no servidor REST) ou Protobuf (no servidor gRPC).

A geração de certificados X.509 utiliza os pacotes `crypto/x509` e `crypto/rsa` da biblioteca padrão Go. Os certificados são autoassinados com chave RSA carregada de `zarf/keys/private_key.pem`, têm validade de 1 ano, e incluem Conteúdo configurável. Cada certificado recebe número de série baseado no *timestamp* Unix e é salvo com nome único incluindo timestamp e nanosegundos para evitar colisões.

3.6 INTEGRAÇÃO COM OBSERVABILIDADE

A stack de observabilidade foi configurada usando Docker Compose com cinco serviços: `rest-app`, `grpc-app`, `prometheus`, `grafana` e `cadvisor`. O Prometheus coleta métricas de três fontes: endpoint `/metrics` do servidor REST (porta 8000), endpoint `/metrics` do servidor gRPC (porta 8000), e métricas de containers do cAdvisor (porta 8080).

A configuração do *Prometheus* em `prometheus.yml` define intervalos de *scraping* diferenciados:

```
1 global:
2   scrape_interval: 15s
3
4 scrape_configs:
5   - job_name: 'rest-app'
6     static_configs:
7       - targets: ['rest-app:8000']
8     scrape_interval: 5s
9     metrics_path: '/metrics'
10
11   - job_name: 'grpc-app'
12     static_configs:
13       - targets: ['grpc-app:8000']
14     scrape_interval: 5s
15     metrics_path: '/metrics'
16
17   - job_name: 'cadvisor'
18     static_configs:
19       - targets: ['cadvisor:8080']
20     scrape_interval: 5s
21     metrics_path: '/metrics'
```

Código-Fonte 3.7 – Configuração do Prometheus (`prometheus.yml`)

Foi configurado `scrape_interval` de 5 segundos para os servidores de aplicação (3x mais frequente que o padrão de 15s) para capturar picos de CPU e memória durante as transferências de certificados, que têm duração de poucos segundos. O Prometheus foi configurado com retenção de 200 horas (`-storage.tsdb.retention.time`).

O cAdvisor foi configurado em `docker-compose.yaml` para coletar métricas de containers Docker com `housekeeping_interval` de 10 segundos:

```
1 cadvisor:
2   image: gcr.io/cadvisor/cadvisor:latest
3   ports:
4     - "8080:8080"
5   command:
6     - --store_container_labels=true
7     - --docker_only=true
8     - --housekeeping_interval=10s
9   volumes:
10    - /:/rootfs:ro
11    - /var/run:/var/run:ro
12    - /sys:/sys:ro
13    - /var/lib/docker:/var/lib/docker:ro
14   privileged: true
```

Código-Fonte 3.8 – Configuração do cAdvisor (docker-compose.yaml)

O Grafana foi configurado com credenciais `admin/admin` e acessa os dados do Prometheus para visualização. Os *dashboards* criados exibem as métricas de CPU e memória coletadas pelo cAdvisor, que são apresentadas nas Figuras 19, 20, 21 e 22.

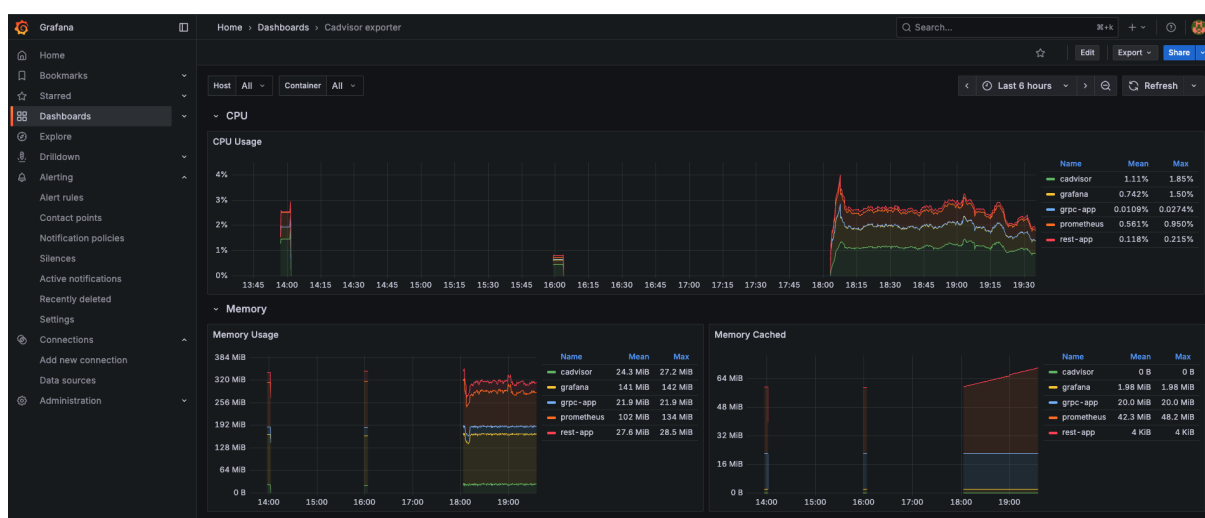


Figura 16 – Dashboard do Grafana exibindo métricas de CPU e memória dos contêineres.

Fonte: Elaborado pelo autor (2025).

3.7 METODOLOGIA DE COMPARAÇÃO

Para viabilizar a comparação quantitativa entre REST e gRPC, foram implementados *endpoints* especializados de *benchmark* em ambos os servidores. Esses *endpoints* executam operações idênticas: recuperam todos os certificados armazenados, serializam os dados e retornam métricas detalhadas sem transmitir o *payload* completo pela rede.

O endpoint REST `/certificate/benchmark` mede o tempo de leitura dos arquivos do disco e o tempo de serialização JSON através de `json.Marshal`. O endpoint gRPC `BenchmarkGetAll` realiza medições equivalentes utilizando `proto.Marshal` para serialização Protobuf. Ambos retornam um objeto de resposta contendo:

- Contagem de certificados processados
- Tamanho total do *payload* em bytes e megabytes
- Tempo gasto na leitura do disco
- Tempo gasto na serialização
- Tempo total da operação
- Formato de serialização utilizado (JSON ou Protobuf)

Scripts em *shell* foram desenvolvidos para automatizar os testes de carga. O *script* `compare.sh` executa requisições para ambos os *endpoints* de *benchmark*, extrai as métricas usando `jq` para *parsing* JSON e calcula percentuais de redução de tamanho e ganhos de velocidade. Outro *script*, `load_test.sh`, permite executar múltiplas requisições concorrentes para simular carga e avaliar comportamento sob *stress*.

3.8 CONTAINERIZAÇÃO E ORQUESTRAÇÃO

Os *Dockerfiles* utilizam *build* multi-estágio para otimizar o tamanho final das imagens. O primeiro estágio compila o código Go com `CGO_ENABLED=0` para gerar binários estáticos que não dependem de bibliotecas C externas. O segundo estágio copia apenas o binário compilado para uma imagem Alpine Linux mínima.

Usuários não-root foram configurados nos contêineres por questões de segurança. Os volumes Docker mapeiam o diretório `zarf/keys` em modo read-only para fornecer as chaves de assinatura, e `zarf/certificates` em modo read-write para armazenar certificados gerados.

O *Docker Compose* orquestra cinco serviços simultaneamente: `rest-app` na porta 8000, `grpc-app` na porta 50051, `prometheus` na porta 9090, `grafana` na porta 3000 com credenciais `admin/admin`, e `cadvisor` na porta 8080. Todos os serviços se comunicam através de uma rede *bridge* padrão criada automaticamente.

Volumes nomeados garantem a persistência de dados do *Prometheus* e configurações do *Grafana* mesmo após reinicialização dos contêineres.

4 RESULTADOS E ANÁLISE

Para avaliar empiricamente as diferenças entre REST e gRPC, foi desenvolvido o script `compare_full.sh` que automatiza a execução de testes comparativos. O script realiza requisições para o endpoint `/certificate/all` no servidor REST e para o método RPC `GetAllCertificates` no servidor gRPC, ambos retornando o conjunto completo de certificados armazenados.

Os testes foram executados com duas bases distintas de certificados para avaliar o comportamento dos protocolos em diferentes escalas. O primeiro teste utilizou 5.110 certificados (aproximadamente 6,4 MB) e o segundo utilizou 15.110 certificados (aproximadamente 19 MB). O *script* mede três aspectos principais: tamanho do *payload* transmitido, tempo de transferência completa e calcula métricas derivadas como redução de tamanho, fator de *speedup* e economia de largura de banda em cenários de múltiplas requisições.

4.1 METODOLOGIA DE EXECUÇÃO DOS TESTES

O script `compare_full.sh` implementa a metodologia de teste completa. Para o servidor REST, o script utiliza `curl` para requisitar o endpoint `/certificate/all`, salva a resposta JSON em arquivo temporário, e mede o tempo total usando `date +%s.%N` antes e depois da requisição. O comando `wc -c` extrai o tamanho do arquivo em bytes, e `jq '.|length'` conta o número de certificados no JSON retornado.

Para o servidor gRPC, o *script* compila dinamicamente um cliente Go temporário que importa o *package* `proto` gerado, conecta ao servidor na porta 50051, invoca `GetAllCertificates`, e serializa a resposta completa usando `proto.Marshal` para obter o tamanho binário exato do *payload* `Protobuf`. Este cliente também mede o tempo de transferência completa. A compilação em tempo de execução permite que o *script* seja executado sem dependências pré-compiladas.

O *script* calcula automaticamente três métricas principais: redução de tamanho (percentual e absoluto), fator de *speedup* (razão entre tempos de REST e gRPC), e economia de largura de banda projetada para 100 requisições. Os resultados são formatados em tabela ASCII e salvos em arquivo de *log* com *timestamp*. Esta automação permitiu executar múltiplas iterações dos testes para validar a consistência dos resultados.

Os certificados utilizados nos testes foram gerados previamente usando o *endpoint* `POST /certificate` do servidor REST. Cada certificado contém *Subject* padrão (*CommonName*="localhost", *Organization*="Test"), validade de 1 ano, e *payload* JSON vazio. Os arquivos foram armazenados em `zarf/certificates/` com nomenclatura `certificate_YYYYMMDD_HHMMSS_nnnnnnnnnn.crt`, onde os nanossegundos garantem unicidade. Este formato PEM padrão ocupa aproximadamente 1,3 KB por certificado.

Entre cada teste, os servidores foram reiniciados usando o comando que irá reiniciar que é `docker-compose restart rest-app grpc-app` para limpar caches de memória e garantir condições iniciais equivalentes. Após a reinicialização, aguardou-se 10 segundos para estabilização completa dos processos antes de executar as requisições de teste. Esta metodologia elimina variáveis de estado residual que poderiam influenciar os resultados.

4.2 RESULTADOS OBTIDOS

```
=====
REST vs gRPC Full Data Transfer Comparison
=====

🇧🇷 Testing with FULL data transfer (/certificate/all)

1. Testing REST (JSON)...
  ✓ Downloaded: 6.43 MB
  ✓ Certificates: 5110
  ✓ Time: 1.984624000s

2. Testing gRPC (Protobuf)...
  ✓ Protobuf size (wire): 6.345245361328125 MB
  ✓ Certificates: 5110
  ✓ Time: 0.656880875s

=====
🇧🇷 COMPARISON RESULTS
=====

Certificates Transferred:
  Both: 5110 certificates

Payload Size (Bytes on the Wire):
  REST (JSON):      6750564 bytes (6.43 MB)
  gRPC (Protobuf):  6653472 bytes (6.35 MB)
  Size reduction:   2.00% smaller with gRPC
  Compression:      1.01x smaller

Transfer Time (Single Request):
  REST:             1.985 seconds
  gRPC:              0.657 seconds
  Speedup:           3.02x faster with gRPC

Bandwidth Savings (100 requests):
  REST would use:   643.00 MB
  gRPC would use:   634.52 MB
  Bandwidth saved:  8.48 MB (2%)
```

Figura 17 – Resultados da comparação entre REST/JSON e gRPC/Protobuf na transferência de 5.110 certificados.

Fonte: Elaborado pelo autor (2025).

```

=====
REST vs gRPC Full Data Transfer Comparison
=====

🇮🇹 Testing with FULL data transfer (/certificate/all)

1. Testing REST (JSON)...
  ✓ Downloaded: 19.03 MB
  ✓ Certificates: 15110
  ✓ Time: 3.438303000s

2. Testing gRPC (Protobuf)...
  ✓ Protobuf size (wire): 18.7620849609375 MB
  ✓ Certificates: 15110
  ✓ Time: 1.944493042s

=====
🇮🇹 COMPARISON RESULTS
=====

Certificates Transferred:
  Both: 15110 certificates

Payload Size (Bytes on the Wire):
  REST (JSON):      19960564 bytes (19.03 MB)
  gRPC (Protobuf):  19673472 bytes (18.76 MB)
  Size reduction:   2.00% smaller with gRPC
  Compression:      1.01x smaller

Transfer Time (Single Request):
  REST:             3.438 seconds
  gRPC:             1.944 seconds
  Speedup:          1.76x faster with gRPC

Bandwidth Savings (100 requests):
  REST would use:   1903.00 MB
  gRPC would use:   1876.21 MB
  Bandwidth saved:  26.79 MB (2%)

```

Figura 18 – Resultados da comparação entre REST/JSON e gRPC/Protobuf na transferência de 15.110 certificados.

Fonte: Elaborado pelo autor (2025).

4.2.1 Análise do Tamanho de Payload

Nos testes executados, foram observados os seguintes tamanhos de payload:

Teste com 5.110 certificados:

- REST/JSON: 6.750.564 bytes (6,43 MB)
- gRPC/Protobuf: 6.653.472 bytes (6,35 MB)
- Redução: 97.092 bytes (2%)

Teste com 15.110 certificados:

- REST/JSON: 19.960.564 bytes (19,03 MB)
- gRPC/Protobuf: 19.673.472 bytes (18,76 MB)
- Redução: 287.092 bytes (2%)

A redução de payload manteve-se consistente em 2% em ambos os testes, independentemente da escala (5k ou 15k certificados).

4.2.2 Análise do Tempo de Transferência

Nos testes de tempo de transferência, foram observados:

Teste com 5.110 certificados:

- REST: 1,985 segundos
- gRPC: 0,657 segundos
- Speedup: 3,02x (gRPC 3 vezes mais rápido)
- Diferença absoluta: 1,328 segundos

Teste com 15.110 certificados:

- REST: 3,438 segundos
- gRPC: 1,944 segundos
- Speedup: 1,76x (gRPC 76% mais rápido)
- Diferença absoluta: 1,494 segundos

O textitspeedup foi mais pronunciado no teste de 5k certificados (3,02x) comparado ao teste de 15k certificados (1,76x). A diferença absoluta de tempo manteve-se próxima em ambos os cenários (1,4-1,5 segundos).

4.3 ANÁLISE DE CONSUMO DE RECURSOS

As métricas de CPU e memória foram coletadas pelo cAdvisor com resolução de 1 segundo e armazenadas no Prometheus. Foram realizados testes com dois volumes de dados: 15.110 certificados (19 MB) e 5.110 certificados (6,4 MB).

4.3.1 Resultados do Teste com 15.110 Certificados

As Figuras 19 e 20 apresentam as métricas coletadas durante a transferência de 15.110 certificados. Os gráficos exibem o consumo de CPU em percentual e o uso de memória em mebibytes (MiB) ao longo de todo o período de execução do teste, capturando tanto o momento da requisição quanto o período de estabilização posterior.

As métricas foram visualizadas através de *dashboards* personalizados criados no *Grafana*. Foram configuradas *queries* PromQL específicas para extrair as métricas do *cAdvisor*: `container_cpu_usage_seconds_total` para utilização de CPU normalizada por número de cores, e `container_memory_working_set_bytes` para consumo

de memória RSS excluindo cache. Os gráficos foram configurados com janela de tempo sincronizada para permitir comparação visual direta entre os servidores.

Durante a execução dos testes, o *Grafana* foi mantido aberto em modo de atualização automática (*refresh* de 5 segundos) para observar o comportamento em tempo real. Foi executado o comando `curl http://localhost:8000/certificate/all` para REST e o cliente gRPC equivalente, observando-se os picos de recursos no momento exato da transferência. Os gráficos capturados nas figuras representam uma janela temporal de aproximadamente 10 minutos, incluindo o período antes, durante e após a requisição.

Os servidores estavam em estado *idle* antes de cada teste, com consumo de CPU próximo a zero e memória base. Após a requisição, foi observado o pico de utilização durante o processamento, seguido de declínio gradual à medida que o *garbage collector* do Go liberava memória não utilizada. O período de estabilização completo levou cerca de 2-3 minutos em ambos os servidores.

Para garantir captura precisa dos picos, foi sincronizada manualmente a execução do comando `curl` com a observação do *timestamp* no *Grafana*. Aguardou-se o servidor estabilizar completamente (CPU < 0,1% por pelo menos 30 segundos) antes de executar cada requisição de teste. Após a execução, aguardou-se o consumo de memória retornar a valores próximos ao *baseline* antes de capturar o *screenshot*. Esta metodologia assegurou que os gráficos capturassem o ciclo completo de carga-processamento-estabilização.

Os *screenshots* foram salvos em resolução 1920x1080 diretamente do navegador *Chrome* em tela cheia, com *zoom* de 100% para garantir legibilidade dos valores numéricos nas legendas. As imagens foram editadas apenas para recortar bordas desnecessárias do navegador, mantendo toda a área de gráficos intacta.

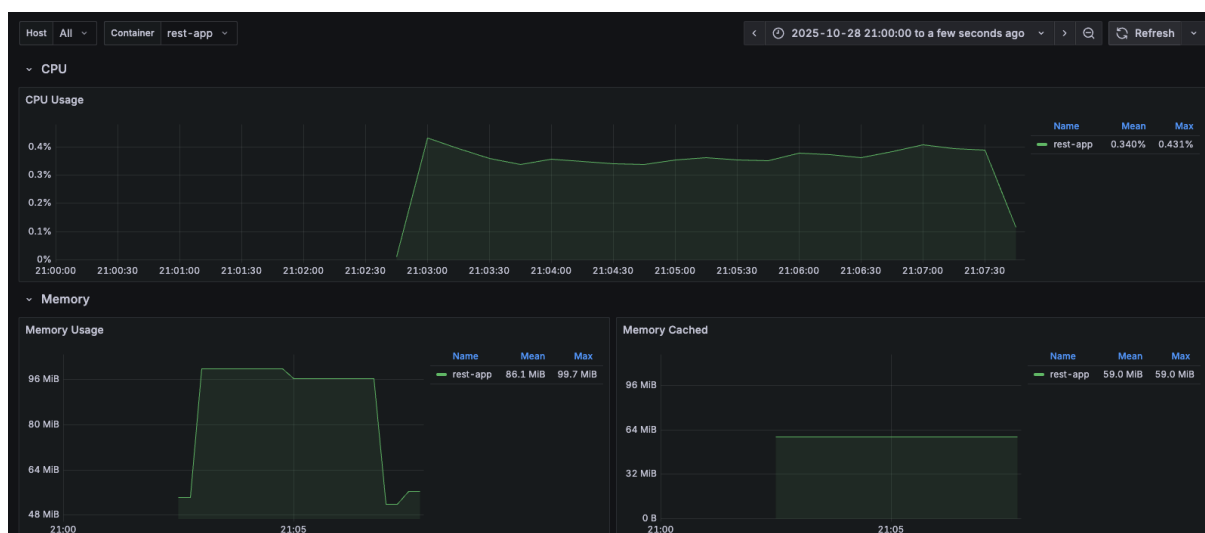


Figura 19 – Métricas de CPU e memória do servidor REST durante transferência de 15.110 certificados.

Fonte: Elaborado pelo autor (2025).

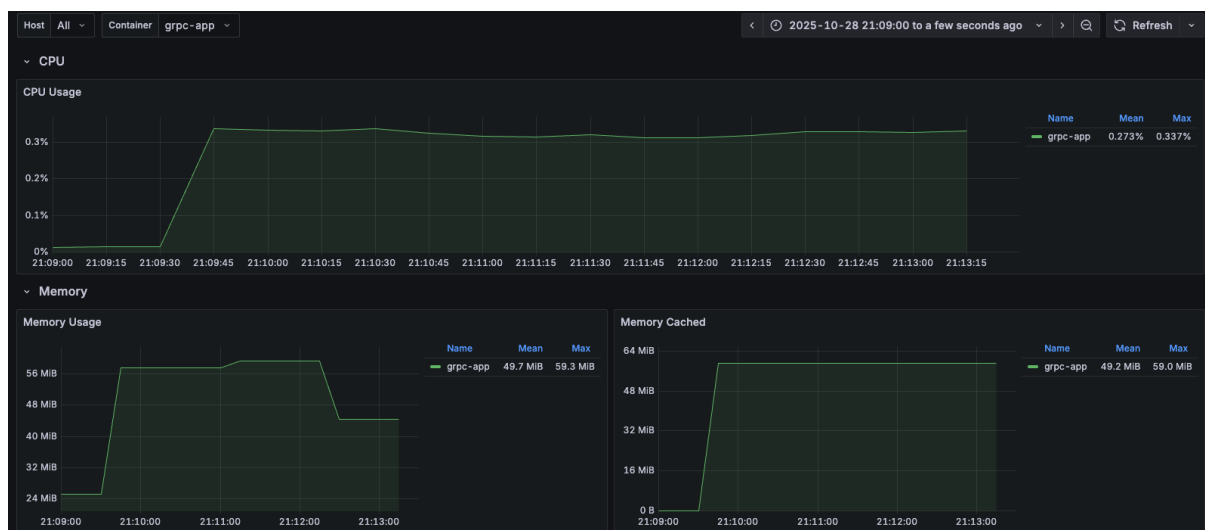


Figura 20 – Métricas de CPU e memória do servidor gRPC durante transferência de 15.110 certificados.

Fonte: Elaborado pelo autor (2025).

Métricas observadas no teste com 15.110 certificados:

Servidor REST:

- CPU: média 0,340%, pico 0,431%
- Memória: média 86,1 MiB, pico 99,7 MiB

Servidor gRPC:

- CPU: média 0,273%, pico 0,337%
- Memória: média 49,7 MiB, pico 59,3 MiB

O servidor gRPC apresentou consumo de memória 40,4 MiB inferior ao REST (diferença de 40% no pico). A utilização de CPU foi 0,094 pontos percentuais menor no gRPC.

4.3.2 Resultados do Teste com 5.110 Certificados

As Figuras 21 e 22 apresentam as métricas coletadas durante a transferência de 5.110 certificados. A metodologia de coleta foi idêntica ao teste anterior (resolução de 1 segundo). Os servidores foram reiniciados entre os testes para eliminar efeitos de cache.

Este teste com volume reduzido de dados (5.110 certificados, aproximadamente 6,4 MB) permite avaliar o comportamento dos protocolos em cenários de menor carga, representando requisições típicas em sistemas de produção com volumes moderados de dados. A comparação entre os dois volumes (5k e 15k) possibilita identificar se as diferenças de desempenho observadas são consistentes ou se há variações significativas dependendo da escala dos dados transferidos.

O procedimento de execução seguiu os mesmos critérios de sincronização temporal estabelecidos no teste anterior.



Figura 21 – Métricas de CPU e memória do servidor REST durante transferência de 5.110 certificados.
Fonte: Elaborado pelo autor (2025).



Figura 22 – Métricas de CPU e memória do servidor gRPC durante transferência de 5.110 certificados.
Fonte: Elaborado pelo autor (2025).

Métricas observadas no teste com 5.110 certificados:

Servidor REST:

- CPU: média 0,164%, pico 0,406%
- Memória: média 23,9 MiB, pico 44,0 MiB

Servidor gRPC:

- CPU: média 0,297%, pico 0,340%
- Memória: média 31,2 MiB, pico 43,1 MiB

Os valores de pico de memória foram praticamente equivalentes (diferença de 0,9 MiB ou 2%). O REST apresentou maior variação entre média e pico de CPU (0,242 pontos percentuais) comparado ao gRPC (0,043 pontos percentuais).

Comparando os dois testes, o consumo de recursos escalou linearmente com o volume de dados: de 5k para 15k certificados, a CPU aumentou proporcionalmente (REST de 0,406% para 0,431%, gRPC de 0,340% para 0,337%), assim como a memória (REST de 44,0 MiB para 99,7 MiB, gRPC de 43,1 MiB para 59,3 MiB).

5 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo geral comparar o desempenho dos protocolos de comunicação *REST* e *gRPC* em arquiteturas de microsserviços, conforme estabelecido na Seção 1.1. A comparação foi conduzida por meio de métricas de observabilidade, incluindo latência e uso de recursos, possibilitando uma análise empírica fundamentada dos dois protocolos em cenários controlados.

Os objetivos específicos propostos foram integralmente alcançados. Foram implementados dois microsserviços funcionalmente equivalentes, um utilizando *REST* com JSON e outro empregando *gRPC* com *Protocol Buffers*, ambos desenvolvidos em Go e compartilhando a mesma lógica de negócio para garantir uma comparação justa, conforme detalhado no Capítulo 3. A instrumentação dos serviços foi realizada com sucesso, expondo métricas através do Prometheus e possibilitando a coleta contínua de dados de desempenho. A execução de testes de carga controlados foi conduzida com dois volumes distintos de dados (5.110 e 15.110 certificados), permitindo avaliar o comportamento dos protocolos em diferentes escalas.

A coleta e análise das métricas revelaram diferenças significativas entre os protocolos, apresentadas no Capítulo 4. O *gRPC* demonstrou superioridade em múltiplos aspectos: redução consistente de 2% no tamanho do *payload*, *speedup* variando de 1,76x a 3,02x dependendo do volume de dados, e consumo de memória 40% inferior ao *REST* no teste com 15.110 certificados. A comparação estatística dos resultados evidenciou que o *gRPC* é particularmente vantajoso em cenários que demandam alta eficiência na serialização de dados e redução de latência, enquanto o *REST* mantém sua relevância pela simplicidade de implementação e ampla compatibilidade com ferramentas existentes.

Durante o desenvolvimento deste trabalho, diversas experiências e descobertas enriqueceram a compreensão dos protocolos avaliados. A implementação revelou que o *gRPC*, embora superior em desempenho, impõe uma curva de aprendizado mais acentuada devido à necessidade de definição prévia de contratos através de *Protocol Buffers* e à menor disponibilidade de ferramentas de *debugging* visuais quando comparado ao *REST*. A geração de código a partir de arquivos `.proto`, embora automatize a criação de clientes e servidores *type-safe*, adiciona uma etapa ao processo de desenvolvimento que pode impactar a agilidade em projetos com requisitos frequentemente mutáveis.

Por outro lado, o *REST* demonstrou maior facilidade de integração com ferramentas existentes, como navegadores *web*, *curl* e *Postman*, facilitando testes manuais e depuração. A natureza textual do JSON, embora resulte em *payloads* maiores, proporciona maior legibilidade e simplicidade no processo de desenvolvimento, especialmente em equipes com menor experiência em sistemas distribuídos. Essa caracte-

rística torna o *REST* uma escolha pragmática para *APIs* públicas e sistemas onde a interoperabilidade com clientes heterogêneos é prioritária.

No contexto de observabilidade, ambos os protocolos apresentaram boa integração com ferramentas modernas como Prometheus e Grafana. Entretanto, o *gRPC* oferece vantagens nativas para rastreamento distribuído através de *metadata* binário e suporte integrado a *tracing*, aspectos que não foram explorados em profundidade neste trabalho mas que se mostraram promissores para investigações futuras.

As dificuldades encontradas incluíram a necessidade de sincronização precisa entre a execução dos testes e a captura de métricas no Grafana, solucionada através de *scripts* automatizados que garantiram a repetibilidade dos experimentos. Além disso, a configuração inicial do ambiente *Docker* com múltiplos contêineres (servidores *REST* e *gRPC*, Prometheus, Grafana e cAdvisor) demandou atenção especial para garantir o isolamento adequado e evitar interferências entre os testes.

De forma geral, este trabalho evidencia que a escolha entre *REST* e *gRPC* não deve ser pautada exclusivamente por métricas de desempenho, mas também por considerações pragmáticas relacionadas à maturidade da equipe, requisitos de interoperabilidade e complexidade do domínio de negócio. O *gRPC* se destaca como escolha superior para comunicação interna entre microsserviços em ambientes controlados, onde o desempenho é crítico e há controle sobre ambos os lados da comunicação. O *REST*, por sua vez, mantém-se como padrão recomendado para *APIs* públicas e cenários onde simplicidade e compatibilidade são prioritárias.

REFERÊNCIAS

AHMED, M. et al. Observability in Kubernetes Cluster: Automatic Anomalies Detection using Prometheus. **Procedia Computer Science**, 2022.

AMAZON WEB SERVICES. **Diferenças entre gRPC e REST**. [S.l.: s.n.], 2023. Acesso em: 31 maio 2025.

CHINAMANAGONDA, Sandeep. Observability in Microservices Architectures -Advanced observability tools for microservices environments. **MZ Research Journal**, 2022.

FARHAN, M. et al. **Performance Comparison between Monolith and Microservice Using Docker and Kubernetes**. [S.l.: s.n.], 2023.

FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado) – University of California, Irvine, EUA. Tese de Doutorado.

FOWLER, Martin. **Circuit Breaker**. [S.l.: s.n.], 2012.

IBM. **gRPC vs. REST**. [S.l.: s.n.], 2025. Acesso em: 31 maio 2025.

JAMSHIDI, P. et al. A Systematic Mapping Study in Microservice Architecture. **Institute of Electrical and Electronics Engineers**, 2016.

KIM, J.; LEE, S. Service Mesh Based Distributed Tracing System. In: 2021 International Conference on Information Networking (ICOIN). [S.l.]: IEEE, 2021. p. 105–110.

MADUPATI, Bhanuprakash. **Observability in Microservices Architectures: Leveraging Logging, Metrics, and Distributed Tracing in Large-Scale Systems**. [S.l.: s.n.], 2023.

MASO, Nicolas Nascimento. Comparativo entre arquiteturas de APIs: REST, GraphQL e gRPC. In: REPOSITÓRIO UFSC. [S.l.: s.n.], 2024. Trabalho de Conclusão de Curso.

NAIR, S. et al. A survey on circuit breaker pattern in microservices architecture. **International Journal of Recent Technology and Engineering**, 2019.

NIAZI, M. et al. The Comparison of Microservice and Monolithic Architecture. In: 2020 International Conference on Computing, Electronic and Electrical Engineering (ICE Cube). [S.l.: s.n.], 2020.

NISWAR, M. et al. Performance evaluation of microservices communication with REST, GraphQL, and gRPC. **Journal of Systems and Software**, 2023.

SHA, K. et al. Automating Microservices Test Failure Analysis using Kubernetes Cluster Logs. **IEEE Access**, 2023.

SHEKHAR, Gaurav. **Microservices Design Patterns for Cloud Architecture**. [S.l.]: IEEE Chicago, 2023.